

青宸智汇——大学生创业智能体平台

设计说明书

文档版本：V1.0

编制日期：2026年8月

密级：内部

本设计说明书详细描述了“青宸智汇——大学生创业智能体平台”的软件架构设计、模块设计、接口设计、界面设计、数据结构设计及运行维护设计方案。文档面向项目开发人员、技术评审人员及系统维护人员，旨在为系统的开发、测试和部署提供完整的技术依据和设计参考。

1 引言

1.1 编写目的

本设计说明书旨在全面、系统地描述“青宸智汇——大学生创业智能体平台”的软件设计方案。文档面向项目经理、系统架构师、前端开发工程师、后端开发工程师、测试工程师及运维人员，为项目的开发实施、质量保证和后续维护提供完整的技术指导。

通过本说明书，开发团队成员可以清晰地理解系统的整体架构、各功能模块的职责划分、核心数据结构的设计方案、接口规范以及界面交互逻辑。同时，文档也为项目评审委员会提供技术评审依据，确保系统设计的合理性、可行性和可维护性。

本说明书的编写遵循软件工程规范，涵盖从系统总体架构到模块级设计的各个层面，力求做到内容详实、逻辑清晰、技术描述准确。文档中涉及的代码示例和接口定义均与实际实现保持一致，可直接作为开发参考。

阅读本文档前，建议读者具备以下技术背景知识：React前端开发框架的基本概念、TypeScript类型系统、微前端架构原理、状态管理库（Zustand）的使用方法、以及人工智能大语言模型的基本交互方式。具备以上知识将有助于更好地理解文档中的技术方案和设计决策。

1.2 项目背景

当前，全国高校创新创业教育蓬勃发展，每年有数百万大学生参与各类创新创业竞赛和创业实践。然而，大学生创业团队普遍面临市场分析能力不足、商业计划撰写困难、项目管理经验匮乏、资源对接渠道有限等痛点。传统的创业辅导方式依赖人工指导，效率低、覆盖面窄，难以满足大规模创业教育的需求。

随着人工智能技术的突破性发展，大语言模型（LLM）在自然语言理解、内容生成、逻辑推理等方面展现出强大能力。将AI技术应用于大学生创业辅导领域，有望实现创业指导的智能化、个性化和规模化。基于这一判断，“青宸智汇——大学生创业智能体平台”项目应运而生。

青宸智汇平台的核心创新在于将AI智能体技术深度融入创业全流程，构建了四个专业化的AI角色——探路者AI、军师AI、工匠AI和管家AI，分别对应创业过程中的市场调研、战略规划、产品制作和项目管理四个关键环节。每个AI角色都具备专业化的知识体系和工具链，能够为大学生创业者提供从市场分析到项目落地的全链路智能服务。

项目采用React 19 + TypeScript 6 + Vite 8 + Ant Design 6的前端技术栈，结合qiankun微前端架构实现模块化部署，使用Zustand进行轻量级状态管理，并集成Three.js实现3D数据可视化。整个系统约29561行源代码，涵盖148个源文件，是一个功能完善、架构清晰的中大型前端应用。

本平台的开发团队由资深前端工程师和AI应用专家组成，在深入调研大学生创业需求的基础上，设计了贴合实际使用场景的功能模块和交互流程。平台已在多所高校进行试点使用，获得了用户的积极反馈，有效提升了大学生创业项目的质量和效率。

1.3 术语定义

为确保文档表述的一致性和准确性，本节对文档中使用的专业术语进行定义和解释。

1.3.1 AI智能体 (AI Agent)

AI智能体是指基于大语言模型构建的、具备特定领域知识和工具调用能力的智能服务实体。在本平台中，AI智能体以“角色”的形式存在，每个角色拥有独立的知识体系、工具集和交互界面。用户通过与AI角色进行多轮对话，获取专业化的创业指导服务。

1.3.2 微前端 (Micro Frontend)

微前端是一种将大型前端应用拆分为多个小型、独立部署的子应用的架构模式。本平台采用qiankun框架实现微前端，支持各功能模块的独立开发、测试和部署，降低模块间的耦合度，提升开发效率和可维护性。

1.3.3 Skill库

Skill库是平台内置的技能管理模块，包含一系列预设的AI技能模板。每个Skill定义了特定场景下的AI交互流程、提示词模板和输出格式。用户可以浏览、启用和管理这些技能，快速获得针对性的AI服务。

1.3.4 MCP (Model Context Protocol)

MCP即模型上下文协议，是一种标准化的AI模型与外部工具的通信协议。本平台通过MCP协议集成各类外部数据源和工具服务，使AI智能体能够动态获取实时数据、调用外部API，从而扩展AI的能力边界。

1.3.5 BMC (Business Model Canvas)

商业模式画布，是一种用于描述和分析商业模式的可视化工具。本平台的军师AI模块实现了BMC的数字化建模，包含九大模块：客户细分、价值主张、渠道通路、客户关系、收入来源、核心资源、关键业务、重要合作和成本结构。

1.3.6 Tool Calling (工具调用)

Tool Calling是大语言模型的一种能力，允许模型在对话过程中动态调用外部工具函数。本平台的AI对话服务支持Tool Calling，使AI能够根据用户需求自动选择并调用相应的工具函数，获取实时数据或执行特定操作。

1.3.7 思维导图 (Mind Map)

思维导图是一种以树形结构组织信息的可视化工具。本平台自研了思维导图编辑器组件，支持大纲模式和导图模式两种视图，并实现了自研的树形布局算法，支持多格式导出。

1.3.8 Token

Token是大语言模型处理文本的基本单位，一个Token通常对应一个词或几个字符。本平台的用量统计模块实时监控AI调用的Token消耗量，帮助用户了解和管理API使用成本。

1.4 参考资料

本设计说明书的编写参考了以下技术文档和规范标准：

1. React 19官方文档——React框架的最新API文档和最佳实践指南。
2. TypeScript 6语言规范——TypeScript类型系统的官方规范文档。
3. Vite 8构建工具文档——前端构建工具Vite的配置和使用指南。
4. Ant Design 6组件库文档——Ant Design企业级UI组件库的API文档。
5. Zustand状态管理库文档——轻量级React状态管理库的使用指南。
6. qiankun微前端框架文档——qiankun框架的接入和配置文档。
7. Three.js 3D图形库文档——WebGL 3D渲染库的API文档。
8. ECharts数据可视化库文档——百度开源图表库的使用文档。
9. GB/T 8567-2006《计算机软件文档编制规范》——中国国家标准，规定了软件文档的编制要求。
10. OpenAI API参考文档——大语言模型API的接口规范和调用指南。
11. Model Context Protocol (MCP) 规范——MCP协议的官方规范文档。
12. 《软件工程——实践者的研究方法》（Roger S. Pressman著）——软件工程经典教材。

2 系统概述

2.1 系统目标

青宸智汇平台的核心目标是构建一个面向大学生创业者的智能辅助平台，通过AI智能体技术为创业全流程提供专业化的智能服务。具体目标包括以下方面：

第一，构建四个专业化AI智能体角色，覆盖创业过程中市场调研、战略规划、产品制作和项目管理四大关键环节，为大学生创业者提供全链路、一站式的智能指导服务。

第二，实现丰富多样的数据可视化能力，通过自研SVG图表组件库和Three.js 3D渲染技术，将抽象的创业数据和分析结果以直观、美观的方式呈现，帮助用户快速理解和决策。

第三，提供完善的工具箱体系，包括Skill技能库、MCP协议集成、自动化触发引擎、知识库检索、应用中心和用量统计等模块，扩展AI智能体的能力边界，满足不同场景的使用需求。

第四，采用先进的微前端架构和现代化前端技术栈，确保系统的可扩展性、可维护性和高性能。系统支持模块化开发、独立部署和动态加载，为后续功能扩展奠定架构基础。

第五，打造优秀的用户体验，通过精心设计的界面布局、流畅的交互流程和美观的视觉设计，降低用户使用门槛，让非技术背景的大学生也能轻松使用平台的各项功能。

第六，实现思维导图编辑器等特色功能模块，为用户提供创新的可视化思考工具，辅助创业过程中的信息整理和方案规划。

2.2 系统功能概述

青宸智汇平台的功能体系由四大AI智能体模块、工具箱模块、数据服务模块和思维导图编辑器模块构成，各模块功能概述如下：

2.2.1 探路者AI模块

探路者AI是平台的市场分析智能体，专注于为大学生创业者提供市场调研和机会评估服务。该模块包含市场分析、竞品调研、趋势洞察和机会评估四个功能面板。市场分析面板通过雷达图、柱状图等可视化组件展示市场规模、增长率和竞争格局等关键指标。竞品调研面板以矩阵图和对比表格的形式呈现竞争对手的优劣势分析。趋势洞察面板利用趋势曲线和面积图展示行业发展趋势和市场热点。机会评估面板通过漏斗图和仪表盘直观展示创业机会的评估结果和优先级排序。

2.2.2 军师AI模块

军师AI是平台的战略规划智能体，为创业者提供商业模式设计和项目规划服务。该模块包含需求分析、商业模式画布、风险矩阵、融资规划和项目计划五个功能面板。需求分析面板帮助用户梳理创业需求，识别核心痛点和目标用户。商业模式画布面板实现了BMC九宫格的可视化编辑，支持实时数据联动和导出。风险矩阵面板通过矩阵图展示风险的严重程度和发生概率，辅助风险决策。融资规划面板提供融资阶段规划和资金需求测算。项目计划面板生成项目里程碑和任务分解结构。

2.2.3 工匠AI模块

工匠AI是平台的产品制作智能体，协助创业者完成产品原型的设计和文档的撰写。该模块包含项目脚手架、PPT大纲、产品文档、原型描述和原型Demo五个功能面板。项目脚手架面板根据用户需求自动生成项目目录结构和基础代码框架。PPT大纲面板生成商业计划书的演示文稿大纲结构。产品文档面板辅助撰写产品需求文档和技术规格说明。原型描述面板通过文字描述生成产品界面的布局方案。原型Demo面板生成可交互的界面原型预览。

2.2.4 管家AI模块

管家AI是平台的项目管理智能体，负责创业项目的任务管理和进度跟踪。该模块包含任务看板、进度跟踪、资源对接和团队协作四个功能面板。任务看板面板以看板形式展示任务的状态流转，支持拖拽操作。进度跟踪面板通过进度环和甘特图展示项目进度。资源对接面板提供创业资源的匹配和推荐。团队协作面板支持团队成员管理和协作沟通。

2.2.5 工具箱模块

工具箱模块为AI智能体提供能力扩展和资源管理功能。包含Skill技能库管理、MCP协议集成、自动化触发引擎、知识库检索、应用中心、用量统计、消息中心和记忆管理等子模块。Skill库提供预设技能模板的浏览和启用功能。MCP集成支持外部工具服务的动态接入。自动化引擎支持基于条件的事件触发和任务执行。知识库提供全文检索和语义搜索能力。用量统计实时监控Token消耗和API调用情况。消息中心统一管理系统通知和AI消息。记忆管理维护AI对话的上下文和用户偏好。

2.2.6 数据服务模块

数据服务模块为平台提供外部数据支撑，包含行业报告、AI政策和行业数据三个子模块。行业报告模块聚合各行业的市场分析报告和研究数据。AI政策模块收集整理与人工智能相关的政策法规和行业规范。行业数据模块提供各行业的统计数据和指标查询。

2.2.7 思维导图编辑器

思维导图编辑器是平台的自研特色功能，采用自研树形布局算法实现节点的自动排列。支持大纲模式和导图模式两种视图切换，支持多格式导出（PNG、SVG、JSON、Markdown等）。编辑器支持节点的增删改查、拖拽排序、折叠展开、样式自定义等操作，为用户提供灵活的思维整理工具。

2.3 用户角色

青宸智汇平台的用户角色分为以下几类：

2.3.1 创业学生（核心用户）

创业学生是平台的核心用户群体，包括在校大学生和毕业不超过三年的青年创业者。他们使用平台的全部功能，从市场调研到项目落地，依托AI智能体的指导完成创业全流程。该类用户通常具备一定的互联网产品使用经验，但对创业专业知识了解有限，需要平台提供易懂、实用的智能指导。

2.3.2 创业导师

创业导师是高校创业教育教师或创业孵化器的指导老师。他们使用平台了解学生的创业项目进展，查看AI生成的分析报告和建议，并在此基础上提供人工指导和补充建议。导师角色需要能够查看学生项目的全貌和AI分析结果。

2.3.3 系统管理员

系统管理员负责平台的日常运维和配置管理。他们管理用户账号、监控API用量、配置AI模型参数、管理Skill库和MCP接入等。管理员通过用量统计模块和消息中心掌握系统运行状态。

2.3.4 平台开发者

平台开发者是负责系统开发和维护的技术人员。他们通过本文设计说明书了解系统架构和模块设计，进行功能开发、Bug修复和系统优化。开发者需要熟悉React、TypeScript和微前端架构等技术。

2.4 运行环境

青宸智汇平台的运行环境要求如下：

2.4.1 开发环境

操作系统：Windows 11（推荐）或 macOS 13+或 Ubuntu 22.04+。

开发工具：Visual Studio Code 或 Trae IDE，配备ESLint、Prettier等代码质量工具。

运行时：Node.js v20+，包管理器使用npm或pnpm。

浏览器：Chrome 120+（开发调试推荐使用最新版Chrome浏览器）。

版本控制：Git，代码托管于GitHub或GitLab。

2.4.2 生产运行环境

客户端环境：支持Chrome 100+、Edge 100+、Firefox 100+、Safari 15+等现代浏览器。不支持IE浏览器。

服务端环境：静态资源部署于CDN或Nginx服务器，AI服务通过HTTPS API调用外部大语言模型服务。

网络要求：需要稳定的互联网连接，建议带宽不低于10Mbps。AI对话功能依赖与外部AI服务的网络通信，网络延迟将影响流式响应体验。

2.4.3 硬件要求

客户端：内存4GB以上，支持WebGL的显卡（3D可视化功能需要）。

显示器：分辨率1366x768以上，推荐1920x1080全高清显示。

服务端：根据用户规模配置，初期支持单节点部署，后期可通过CDN和负载均衡实现横向扩展。

3 系统架构设计

3.1 总体架构

青宸智汇平台采用单页应用（SPA）与微前端（Micro Frontend）相结合的架构模式。整体架构分为四个层次：展示层、业务逻辑层、服务层和基础设施层。

展示层负责用户界面的渲染和交互处理，基于React 19组件化架构构建，使用Ant Design 6组件库提供企业级UI基础组件。展示层还包括Three.js 3D渲染引擎和自研SVG图表组件库，用于实现丰富的数据可视化效果。

业务逻辑层是平台的核心层，包含四大AI智能体模块（探路者AI、军师AI、工匠AI、管家AI）、工具箱模块、数据服务模块和思维导图编辑器模块。各业务模块通过qiankun微前端框架实现独立加载和隔离运行，降低模块间耦合度。

服务层负责与外部服务的通信，包括AI对话服务（与大语言模型API通信）、认证服务（用户身份管理）、知识服务（知识库检索）和行业数据服务等。服务层通过统一的HTTP请求封装层进行网络通信，内置请求拦截、错误处理和重试机制。

基础设施层提供全局状态管理（Zustand + Immer）、路由管理（React Router）、构建系统（Vite 8）和工具函数库等基础能力支撑。

3.1.1 SPA架构设计

平台主体采用单页应用（SPA）架构，通过React Router实现客户端路由，页面切换无需整页刷新，提供流畅的用户体验。应用入口在main.tsx中初始化React根组件，挂载App组件并注册全局路由。路由配置采用嵌套路由结构，主路由包含登录页、主布局页和各功能模块页。

SPA架构的优势在于：首次加载后，后续页面切换仅需加载必要的代码块（通过Vite的代码分割功能实现），大幅减少网络请求和数据传输量。同时，客户端路由使得页面状态可以在路由间保持，用户在不同功能模块间切换时无需重新加载数据。

3.1.2 微前端qiankun集成

平台通过qiankun微前端框架实现模块化部署。主应用作为基座（Container），负责全局状态管理、路由分发和子应用生命周期管理。各功能模块作为子应用（SubApp）独立开发、独立构建、独立部署。

qiankun的核心机制包括：应用注册（registerMicroApps）、应用启动（start）、应用加载（load MicroApp）和应用通信（initGlobalState）。主应用在启动时注册所有子应用，根据路由匹配自动加载对应的子应用。子应用通过生命周期函数（bootstrap、mount、unmount）与主应用协作。

微前端架构带来的好处包括：模块解耦——各子应用可独立开发和部署；技术无关——子应用可以使用不同的技术栈（虽然本平台统一使用React）；渐进式重构——可以逐步将旧模块迁移到新架构；独立部署——某个模块更新时无需重新部署整个应用。

```
// qiankun 微前端注册示例
import { registerMicroApps, start } from 'qiankun';
```

```
const apps = [
  {
    name: 'explorer-ai',
    entry: '//localhost:7101',
    container: '#sub-app-container',
    activeRule: '/explorer',
  },
  {
    name: 'strategist-ai',
    entry: '//localhost:7102',
    container: '#sub-app-container',
    activeRule: '/strategist',
  },
  {
    name: 'artisan-ai',
    entry: '//localhost:7103',
    container: '#sub-app-container',
    activeRule: '/artisan',
  },
  {
    name: 'steward-ai',
    entry: '//localhost:7104',
    container: '#sub-app-container',
    activeRule: '/steward',
  },
];

registerMicroApps(apps);
start({ prefetch: true, sandbox: true });
```

3.2 技术选型说明

平台的技术选型基于以下原则：成熟稳定、社区活跃、性能优异、开发效率高、类型安全。各核心技术的选型理由如下：

3.2.1 React 19

React 19是当前最流行的前端UI框架，拥有庞大的社区生态和丰富的第三方组件库。React 19引入了多项改进，包括改进的并发渲染（Concurrent Rendering）、自动批处理（Automatic Batching）、Suspense for Data Fetching等特性，提升了应用的性能和开发体验。选择React 19作为UI框架，使平台能够利用其成熟的组件化模式、Hooks机制和丰富的生态资源，快速构建高质量的用户界面。

3.2.2 TypeScript 6

TypeScript为JavaScript添加了静态类型系统，在编译阶段即可发现类型错误，大幅提升代码的可靠性和可维护性。TypeScript 6在类型推断、模板字面量类型等方面进行了增强。平台全面采用TypeScript编写，所有组件、接口、工具函数均具备完整的类型定义，确保代码的类型安全和IDE智能提示能力。类型系统的使用使得重构更加安全，团队协作更加高效。

3.2.3 Vite 8

Vite是基于ESBuild和Rollup的新一代前端构建工具，相比传统的Webpack，Vite在开发模式下利用浏览器原生ESM模块，实现毫秒级的热更新（HMR），大幅提升开发效率。生产构建使用Rollup进行代码分割和Tree Shaking，生成优化的生产包。Vite 8在插件系统、构建性能和配置体验上进一步优化，是当前React项目的首选构建工具。

3.2.4 Ant Design 6

Ant Design是蚂蚁集团开源的企业级UI组件库，提供丰富的React组件（按钮、表单、表格、弹窗、导航等），设计规范统一，文档完善。Ant Design 6在组件性能、主题定制和可访问性方面进行了优化。平台使用Ant Design作为基础UI组件库，在此基础上通过主题定制和样式覆写实现品牌化的视觉风格。

3.2.5 Zustand + Immer

Zustand是一个轻量级的React状态管理库，相比Redux具有更简洁的API和更小的包体积。Zustand使用Hook模式，天然支持React的函数组件。结合Immer不可变数据库，可以实现以可变语法编写不可变更新，简化状态更新逻辑。平台使用Zustand管理全局状态（用户信息、AI对话状态、应用配置等），每个状态store职责单一，便于测试和维护。

3.2.6 qiankun

qiankun是蚂蚁集团开源的微前端框架，基于single-spa封装，提供了更完善的沙箱隔离、样式隔离和通信机制。选择qiankun作为微前端方案，使平台能够将大型应用拆分为多个独立部署的子应用，实现团队并行开发和独立部署。

3.2.7 Three.js

Three.js是最流行的WebGL 3D图形库，提供丰富的3D渲染能力。平台使用Three.js实现3D仪表盘（Dashboard3D）组件，将关键运营指标以3D球体、立体柱状图等形式呈现，增强数据可视化的视觉冲击力和交互趣味性。

3.2.8 ECharts

ECharts是百度开源的数据可视化图表库，支持折线图、柱状图、饼图、雷达图等多种图表类型。平台在部分数据展示场景中使用ECharts，配合自研SVG图表组件库，形成完整的可视化方案。

3.3 目录结构设计

平台的源代码目录结构遵循功能模块化的组织原则，主要目录结构如下：

```
react-ai-chat/
├── public/                # 静态资源
├── src/
│   ├── components/       # 通用组件
│   │   ├── common/       # 基础通用组件
│   │   ├── charts/       # SVG图表组件库
│   │   ├── layout/       # 布局组件
│   │   └── mindmap/       # 思维导图编辑器
│   ├── modules/          # 业务模块
│   │   ├── explorer/     # 探路者AI
│   │   ├── strategist/   # 军师AI
│   │   ├── artisan/      # 工匠AI
│   │   ├── steward/      # 管家AI
│   │   ├── toolbox/      # 工具箱
│   │   └── data-service/  # 数据服务
│   ├── services/         # 服务层
│   │   ├── ai/           # AI对话服务
│   │   ├── auth/         # 认证服务
│   │   ├── knowledge/    # 知识服务
│   │   └── industry/     # 行业数据服务
│   ├── stores/           # 状态管理
│   ├── types/            # TypeScript类型定义
│   ├── utils/            # 工具函数
│   ├── hooks/            # 自定义Hooks
│   ├── constants/        # 常量定义
│   ├── styles/           # 全局样式
│   ├── App.tsx           # 应用根组件
│   ├── main.tsx          # 应用入口
│   └── router.tsx        # 路由配置
├── package.json
├── tsconfig.json
├── vite.config.ts
└── README.md
```

目录结构的设计原则是：通用组件与业务组件分离、类型定义集中管理、服务层与展示层解耦、每个业务模块自成体系。这种结构使得各模块可以独立开发和维护，也方便后续通过微前端框架进行拆分。

3.4 模块划分

平台的模块划分遵循“高内聚、低耦合”的原则，将系统功能划分为以下几大模块：

3.4.1 AI智能体模块群

AI智能体模块群是平台的核心，包含探路者AI、军师AI、工匠AI和管家AI四个子模块。每个AI子模块包含多个功能面板，每个面板是一个独立的功能单元，负责特定的业务逻辑和UI展示。AI模块群共享统一的对话服务接口和消息处理逻辑，但各模块的领域知识和工具集各自独立。

3.4.2 工具箱模块

工具箱模块提供AI能力扩展和资源管理功能，包含Skill库、MCP配置、自动化引擎、知识库、应用中心、用量统计、消息中心和记忆管理八个功能单元。该模块作为AI智能体的能力底座，为各AI角色提供通用的工具和资源。

3.4.3 数据服务模块

数据服务模块提供外部数据支撑，包含行业报告、AI政策和行业数据三个功能单元。该模块通过服务层与外部数据源通信，为AI智能体提供数据输入。

3.4.4 通用组件模块

通用组件模块包含SVG图表组件库、布局组件、思维导图编辑器组件和基础通用组件。SVG图表组件库是平台自研的轻量级可视化组件集合，包含雷达图、漏斗图、矩阵图、趋势曲线等十余种图表类型。思维导图编辑器是平台的自研特色组件，采用自研树形布局算法。

3.4.5 服务层模块

服务层模块包含AI对话服务、认证服务、知识服务和行业数据服务。服务层通过统一的HTTP封装层与后端API通信，提供请求拦截、错误处理和响应转换等功能。

3.5 状态管理架构

平台采用Zustand作为全局状态管理方案，结合Immer实现不可变数据更新。状态管理架构设计如下：

3.5.1 Zustand Store设计

平台的Zustand Store按业务领域划分为多个独立Store，每个Store管理特定领域的状态。主要的Store包括：

useAuthStore——管理用户认证状态，包括登录状态、用户信息、Token等。该Store提供login、logout、restoreSession等Action，在应用启动时尝试从localStorage恢复会话。

useAIChatStore——管理AI对话状态，包括当前对话列表、消息历史、对话参数（温度、模型等）、流式响应状态等。该Store是AI模块的核心状态管理器，处理消息的发送、接收和渲染。

useToolboxStore——管理工具箱状态，包括Skill列表、MCP配置、自动化规则、知识库索引等。

useAppStore——管理全局应用状态，包括当前激活的AI角色、侧边栏折叠状态、主题配置等。

useMindMapStore——管理思维导图编辑器状态，包括节点树、当前选中节点、布局模式等。

```
// Zustand Store 示例
import { create } from 'zustand';
import { immer } from 'zustand/middleware/immer';
import { devtools, persist } from 'zustand/middleware';

interface AIChatState {
  conversations: Conversation[];
  currentConversationId: string | null;
  messages: Message[];
  isStreaming: boolean;
  modelConfig: ModelConfig;

  sendMessage: (content: string) => Promise<void>;
  receiveStreamChunk: (chunk: string) => void;
  createConversation: (role: AIRole) => void;
  deleteConversation: (id: string) => void;
  setModelConfig: (config: Partial<ModelConfig>) => void;
}

export const useAIChatStore = create<AIChatState>()(
  devtools(
    persist(
      immer((set) => ({
        conversations: [],
        currentConversationId: null,
        messages: [],
        isStreaming: false,
        modelConfig: { model: 'gpt-4o', temperature: 0.7 },

        sendMessage: async (content) => {
          set((state) => {
            state.messages.push({ role: 'user', content });
            state.isStreaming = true;
          });
          // 调用AI服务发送消息...
        },

        receiveStreamChunk: (chunk) => {
          set((state) => {
            const lastMsg = state.messages[state.messages.length - 1];
            if (lastMsg && lastMsg.role === 'assistant') {
              lastMsg.content += chunk;
            } else {
              state.messages.push({ role: 'assistant', content: chunk });
            }
          });
        },
      })),
      { name: 'ai-chat-storage' }
    )
  )
)
```

```
);
```

3.5.2 Immer集成

Immer是JavaScript不可变数据管理库，允许开发者以可变语法编写不可变更新。Zustand通过immer中间件集成Immer，在set回调中可以直接以可变方式修改状态，Immer会在底层生成新的状态对象。这种模式既保证了状态更新的不可变性，又简化了开发者的编码心智负担。

在AI对话场景中，Immer的优势尤为明显。当处理流式响应时，需要不断追加最后一条消息的内容，使用Immer可以直接修改消息对象的content属性，代码清晰简洁。如果不使用Immer，则需要手动展开数组和新对象，代码冗长且容易出错。

3.5.3 状态持久化

部分状态需要持久化到浏览器本地存储，以便在页面刷新后恢复。Zustand通过persist中间件实现状态持久化，支持localStorage和sessionStorage两种存储后端。平台对useAuthStore和useAIChatStore启用了持久化，确保用户的登录状态和对话历史在页面刷新后不丢失。持久化数据通过JSON序列化存储，在应用启动时自动反序列化恢复。

3.6 构建系统

平台使用Vite 8作为构建工具，构建系统配置如下：

3.6.1 开发模式

开发模式下，Vite利用浏览器原生ESM模块支持，按需编译和加载模块，实现毫秒级热更新（HMR）。开发者修改代码后，浏览器几乎实时刷新，无需等待完整编译。Vite的依赖预构建（Dependency Pre-bundling）功能会将node_modules中的CommonJS依赖转换为ESM格式，并缓存到node_modules/.vite目录，进一步提升开发体验。

3.6.2 生产构建

生产构建使用Rollup进行打包。Vite配置了代码分割（Code Splitting），将vendor代码（React、Ant Design等第三方库）和业务代码分离，并按路由进行懒加载分割，减小初始加载体积。Tree Shaking功能自动移除未使用的代码，进一步优化包体积。构建产物输出到dist目录，包含index.html、JS和CSS等静态资源。

```
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
```

```
    '@': path.resolve(__dirname, 'src'),
  },
},
build: {
  rollupOptions: {
    output: {
      manualChunks: {
        'react-vendor': ['react', 'react-dom', 'react-router-dom'],
        'antd-vendor': ['antd', '@ant-design/icons'],
        'three-vendor': ['three'],
        'echarts-vendor': ['echarts'],
      },
    },
  },
  chunkSizeWarningLimit: 1000,
},
server: {
  port: 3000,
  proxy: {
    '/api': {
      target: 'http://localhost:8080',
      changeOrigin: true,
    },
  },
},
});
```

3.6.3 微前端构建

对于qiankun微前端子应用，Vite配置需要额外处理。子应用需要在入口文件导出生命周期函数（bootstrap、mount、unmount），并配置允许跨域访问。Vite的base路径需要设置为子应用的部署路径，确保静态资源的正确加载。

4 AI智能体模块设计

AI智能体模块是青宸智汇平台的核心业务模块，包含探路者AI、军师AI、工匠AI和管家AI四个专业化智能体。每个AI智能体拥有独立的功能面板体系、领域知识库和工具链，通过统一的AI对话服务与用户进行多轮交互。本章详细描述各AI智能体的模块设计方案。

4.1 探路者AI模块设计

探路者AI是平台的市场分析智能体，专注于为大学生创业者提供市场调研和机会评估服务。该模块包含市场分析、竞品调研、趋势洞察和机会评估四个功能面板，每个面板配备丰富的SVG数据可视化组件，帮助用户直观理解市场数据和分析结果。

4.1.1 市场分析面板

市场分析面板是探路者AI的第一个功能面板，用于展示市场整体规模、增长率、竞争格局等关键指标。面板的核心可视化组件包括雷达图、柱状图和环形图。

雷达图组件用于展示市场多维度评估结果，维度包括市场规模、增长潜力、竞争强度、进入门槛、政策支持和资本热度等六个维度。雷达图的每个维度数据由AI根据用户输入的行业和市场信息生成，数据范围为0-100分。雷达图采用SVG绘制，支持悬停高亮、图例切换和动画过渡。

柱状图组件用于展示市场规模的历史数据和预测数据。横轴为时间（年份），纵轴为市场规模（亿元）。柱状图支持正负值显示、数据标签、趋势线叠加等功能。在市场分析场景中，柱状图直观呈现市场的发展趋势和增长幅度。

环形图组件用于展示市场细分领域的占比分布。环形图以圆环形式展示各细分市场的份额比例，中心显示总市场规模。环形图支持悬停高亮、点击筛选和图例联动。

4.1.2 竞品调研面板

竞品调研面板用于展示竞争对手的分析和对比结果。该面板的核心可视化组件包括矩阵图 and 对比表格。

矩阵图组件以二维矩阵的形式展示竞争对手的市场定位。横轴代表“产品差异化程度”，纵轴代表“价格竞争力”。矩阵图将竞争对手分为四个象限：领导者（右上）、挑战者（左上）、跟随者（右下）、利基者（左下）。每个竞争对手以圆形节点表示，节点大小代表市场份额。矩阵图支持节点拖拽、悬停详情和象限标注。

对比表格组件以表格形式展示竞争对手的多维度对比数据，包括公司名称、成立时间、融资轮次、团队规模、核心产品、目标客户、定价策略、优势、劣势等字段。表格支持排序、筛选和导出功能，帮助用户系统化地了解竞争格局。

4.1.3 趋势洞察面板

趋势洞察面板用于展示行业发展趋势和市场热点。该面板的核心可视化组件包括趋势曲线、面积图和时间线。

趋势曲线组件展示行业关键指标随时间的变化趋势。曲线采用平滑贝塞尔曲线绘制，支持多条曲线叠加对比。曲线下方可填充半透明渐变色，形成面积图效果。趋势曲线支持数据点悬停tooltip、缩放和平移操作。

面积图组件在趋势曲线的基础上增加了面积填充效果，更直观地展示数据量的变化幅度。面积图支持渐变填充、堆叠面积和多数据系列展示。

时间线组件以时间轴形式展示行业发展的重要里程碑事件。时间线采用垂直布局，每个事件节点包含时间标签、事件标题和事件描述。时间线支持事件筛选、展开收起和动画进入效果。

4.1.4 机会评估面板

机会评估面板用于展示创业机会的评估结果和优先级排序。该面板的核心可视化组件包括漏斗图、仪表盘和进度环。

漏斗图组件以漏斗形状展示创业机会的筛选流程，从“市场总量”到“可触达市场”到“可服务市场”到“可获得市场”逐层递减。漏斗图每层显示数量和占比，帮助用户理解市场的分层结构。漏斗图采用SVG path绘制，支持动画过渡和悬停高亮。

仪表盘组件以仪表表盘的形式展示机会评分的综合得分。仪表盘指针指向0-100分的刻度，颜色从红到黄到绿渐变，直观表示评分等级。仪表盘支持自定义刻度范围、颜色区间和动画过渡。

进度环组件以环形进度条的形式展示各评估维度的达成情况。每个维度（市场规模、技术可行性、团队匹配度、资金需求、政策支持等）以独立的进度环展示，环内显示百分比数值。多个进度环可以横向排列，形成评估仪表盘效果。

4.1.5 探路者AI数据流设计

探路者AI的数据流遵循“用户输入 -> AI分析 -> 结果渲染”的模式。用户在对话面板中输入行业和市场信息，AI对话服务将用户消息发送至大语言模型API，AI根据预设的探路者角色提示词生成分析结果。分析结果以结构化JSON格式返回，前端根据数据类型选择对应的SVG图表组件进行渲染。

数据流的关键环节包括：消息构建（将用户输入组装为AI对话消息格式）、API调用（通过流式接口发送请求并接收响应）、响应解析（将AI返回的结构化数据解析为图表数据格式）、图表渲染（将数据传递给SVG图表组件进行可视化展示）。整个流程通过Zustand状态管理驱动，确保数据流的单向性和可追踪性。

4.2 军师AI模块设计

军师AI是平台的战略规划智能体，为创业者提供商业模式设计和项目规划服务。该模块包含需求分析、商业模式画布、风险矩阵、融资规划和项目计划五个功能面板。

4.2.1 需求分析面板

需求分析面板帮助用户梳理创业需求，识别核心痛点和目标用户。面板以结构化的卡片布局展示需求分析结果，包括以下内容板块：

目标用户画像——展示目标用户的基本属性（年龄、职业、收入等）和行为特征。AI根据用户输入的创业方向自动生成用户画像，包括用户痛点、需求和期望。

核心痛点分析——以列表形式展示目标用户面临的核心痛点，每个痛点包含痛点描述、影响范围和紧急程度。痛点按紧急程度排序，帮助用户识别最需要解决的问题。

需求优先级矩阵——以矩阵图形式展示需求的优先级评估结果。横轴为“用户价值”，纵轴为“实现难度”。需求按象限分类：快速胜利（高价值低难度）、战略投资（高价值高难度）、便利功能（低价值低难度）、暂不考虑（低价值高难度）。

4.2.2 商业模式画布（BMC）面板

商业模式画布面板是军师AI的核心功能，实现了BMC九宫格的可视化编辑和展示。BMC数据结构定义如下：

```
// 商业模式画布数据结构
interface BusinessModelCanvas {
  id: string;
  projectName: string;
  customerSegments: BMCBlock[];    // 客户细分
  valuePropositions: BMCBlock[];   // 价值主张
  channels: BMCBlock[];             // 渠道通路
  customerRelationships: BMCBlock[]; // 客户关系
  revenueStreams: BMCBlock[];       // 收入来源
  keyResources: BMCBlock[];         // 核心资源
  keyActivities: BMCBlock[];        // 关键业务
  keyPartnerships: BMCBlock[];      // 重要合作
  costStructure: BMCBlock[];        // 成本结构
}

interface BMCBlock {
  id: string;
  title: string;
  content: string;
  priority: 'high' | 'medium' | 'low';
  tags: string[];
}
```

BMC面板的UI采用九宫格布局，每个格子对应一个商业模式要素。格子内显示该要素的内容卡片列表，支持卡片的增删改查和拖拽排序。AI可以根据用户输入的创业方向自动填充BMC各要素的初始内容，用户可以在此基础上进行修改和完善。

BMC面板支持实时预览和导出功能。用户可以一键导出BMC为PNG图片或PDF文档，方便在商业计划书中引用。BMC数据通过Zustand状态管理维护，确保修改的实时同步和持久化。

4.2.3 风险矩阵面板

风险矩阵面板通过矩阵图展示创业风险的评估结果。风险矩阵的横轴为“发生概率”（低、中、高），纵轴为“影响程度”（低、中、高、极高）。每个风险以圆形节点表示，节点大小代表风险的综合评分，节点颜色按风险等级区分（红=高危、橙=中危、绿=低危）。

```
// 风险矩阵算法
interface RiskItem {
  id: string;
```

```
name: string;
description: string;
probability: number;    // 发生概率 0-1
impact: number;         // 影响程度 1-4
category: RiskCategory;
mitigation: string;     // 缓解措施
}

// 风险评分计算
function calculateRiskScore(probability: number, impact: number): number {
    return probability * impact * 25; // 归一化到0-100
}

// 风险等级判定
function getRiskLevel(score: number): 'high' | 'medium' | 'low' {
    if (score >= 60) return 'high';
    if (score >= 30) return 'medium';
    return 'low';
}

// 矩阵定位
function getMatrixPosition(probability: number, impact: number): {x: number, y: number} {
    const col = Math.floor(probability * 3);    // 0-2
    const row = Math.min(Math.floor(impact - 1), 3); // 0-3
    return { x: col, y: row };
}
```

风险矩阵面板还包含风险详情列表，以表格形式展示每个风险的详细信息，包括风险名称、风险描述、发生概率、影响程度、风险评分、风险等级和缓解措施。表格按风险评分降序排列，帮助用户优先关注高风险项。

AI在生成风险矩阵时，会根据用户输入的创业方向和商业模式自动识别潜在风险，并为每个风险建议缓解措施。用户可以手动添加、修改或删除风险项，修改实时反映到矩阵图中。

4.2.4 融资规划面板

融资规划面板为创业者提供融资阶段规划和资金需求测算服务。面板内容包括：

融资阶段规划——展示种子轮、天使轮、A轮、B轮等融资阶段的时间线和资金需求。每个阶段包含融资金额、出让股权比例、资金用途和预期里程碑。

资金需求测算——根据商业模式画布中的成本结构，自动测算各阶段的资金需求。资金需求包括人力成本、运营成本、市场推广成本和技术开发成本等。

融资时间线——以甘特图形式展示融资计划的执行时间线，包括各阶段的启动时间、预计完成时间和关键节点。

4.2.5 项目计划面板

项目计划面板生成项目里程碑和任务分解结构。面板内容包括：

里程碑规划——以时间线形式展示项目的关键里程碑，包括产品发布、用户增长目标、营收目标等。每个里程碑包含目标日期、达成标准和状态标识。

WBS任务分解——以树形结构展示工作分解结构（WBS），将项目分解为可执行的任务包。每个任务包包含负责人、预计工时、开始时间和结束时间。

甘特图——以甘特图形式展示任务的执行时间线和依赖关系。甘特图支持任务拖拽调整、依赖连线显示和关键路径标注。

4.3 工匠AI模块设计

工匠AI是平台的产品制作智能体，协助创业者完成产品原型的设计和文档的撰写。该模块包含项目脚手架、PPT大纲、产品文档、原型描述和原型Demo五个功能面板。

4.3.1 项目脚手架面板

项目脚手架面板根据用户需求自动生成项目目录结构和基础代码框架。用户通过对话描述项目需求（技术栈、功能模块、页面数量等），AI生成完整的项目脚手架方案。

项目脚手架生成逻辑包括以下步骤：第一步，需求解析——AI解析用户输入的自然语言需求，提取技术栈、功能模块和页面结构等关键信息。第二步，模板匹配——根据解析结果匹配预设的项目模板（React项目模板、Vue项目模板、Node.js后端模板等）。第三步，目录生成——根据模板和用户需求生成完整的目录结构树。第四步，代码生成——为关键文件生成基础代码内容（入口文件、路由配置、状态管理初始化等）。第五步，输出展示——以树形目录和代码预览的形式展示生成结果，支持一键下载为ZIP文件。

```
// 项目脚手架数据结构
interface ScaffoldConfig {
  projectName: string;
  template: 'react' | 'vue' | 'node' | 'fullstack';
  typescript: boolean;
  features: {
    router: boolean;
    stateManagement: 'zustand' | 'redux' | 'none';
    uiLibrary: 'antd' | 'mui' | 'none';
    testing: boolean;
    eslint: boolean;
    prettier: boolean;
  };
  pages: PageConfig[];
  modules: string[];
}

interface FileNode {
  name: string;
  type: 'file' | 'directory';
  content?: string;
  children?: FileNode[];
}
```

4.3.2 PPT大纲面板

PPT大纲面板生成商业计划书的演示文稿大纲结构。AI根据用户提供的项目信息和商业模式画布内容，自动生成符合投资人路演标准的PPT大纲。

```
// PPT大纲数据结构
interface PPTOutline {
  title: string;
  slides: PPTSlide[];
}

interface PPTSlide {
  slideNumber: number;
  title: string;
  layout: 'cover' | 'content' | 'image' | 'chart' | 'quote' | 'closing';
  bulletPoints: string[];
  speakerNotes: string;
  suggestedVisual?: string;
}
```

PPT大纲的标准结构包括：封面（项目名称、团队信息、日期）、问题与机会（市场痛点分析）、解决方案（产品介绍）、市场分析（市场规模和增长趋势）、商业模式（BMC概览）、竞争分析（竞品对比）、团队介绍、财务规划（融资需求和资金用途）、发展路线图、总结与愿景等10-15页幻灯片。每页幻灯片包含标题、要点列表、演讲备注和建议配图。用户可以编辑大纲内容，并导出为Markdown或JSON格式。

4.3.3 产品文档面板

产品文档面板辅助撰写产品需求文档（PRD）和技术规格说明。AI根据用户描述的产品功能和需求，生成结构化的产品文档。文档包含以下章节：产品概述、目标用户、功能需求（功能列表、用例描述、界面原型描述）、非功能需求（性能、安全、兼容性）、数据需求（数据模型、数据字典）、接口需求（API定义）和附录。

产品文档面板支持实时编辑和预览，用户可以在AI生成的基础上修改文档内容。文档支持Markdown格式编辑，实时渲染为格式化的文档视图。面板还提供文档版本管理功能，记录每次修改的历史版本。

4.3.4 原型描述面板

原型描述面板通过文字描述生成产品界面的布局方案。用户描述产品页面的功能需求和布局偏好，AI生成详细的界面布局描述，包括页面区域划分、组件类型选择、交互行为定义等。

原型描述的输出格式为结构化的JSON数据，包含页面信息、区域列表和组件定义。每个区域包含区域名称、布局类型（flex/grid）、尺寸比例和包含的组件列表。每个组件定义包含组件类型（按钮、输入框、表格等）、属性配置和交互事件。

4.3.5 原型Demo面板

原型Demo面板根据原型描述数据生成可交互的界面原型预览。面板将结构化的原型描述渲染为真实的UI界面，用户可以在浏览器中直接操作原型，体验产品的交互流程。原型Demo支持页面导航、表单填写、弹窗展示等基本交互行为，帮助用户直观感受产品的最终效果。

4.4 管家AI模块设计

管家AI是平台的项目管理智能体，负责创业项目的任务管理和进度跟踪。该模块包含任务看板、进度跟踪、资源对接和团队协作四个功能面板。

4.4.1 任务看板面板

任务看板面板以看板形式展示任务的状态流转。看板采用三列布局：待办（To Do）、进行中（In Progress）和已完成（Done）。每个任务以卡片形式展示，卡片包含任务标题、负责人、优先级标签、截止日期和任务描述。

任务看板支持拖拽操作，用户可以通过拖拽卡片在不同的列之间移动任务，实现状态流转。拖拽过程中有视觉反馈（卡片阴影、放置区域高亮），拖拽完成后自动更新任务状态。看板还支持任务筛选（按负责人、优先级、标签等）、任务搜索和批量操作。

4.4.2 进度跟踪面板

进度跟踪面板通过进度环和甘特图展示项目进度。进度环以环形进度条形式展示项目整体完成率和各模块完成率。甘特图以条形图形式展示各任务的时间线和完成状态。

进度跟踪面板还包含统计数据卡片，展示关键指标：任务总数、已完成数、进行中数、逾期任务数、本周完成任务数等。这些指标通过热力计组件以仪表盘形式展示，直观反映项目的健康状态。

4.4.3 资源对接面板

资源对接面板提供创业资源的匹配和推荐服务。面板展示各类创业资源，包括资金资源（投资机构、政府补贴）、人才资源（招聘平台、人才推荐）、场地资源（孵化器、众创空间）和服务资源（法务、财税、设计等）。

资源推荐基于AI分析用户的项目需求和阶段，自动匹配适合的资源。每个资源卡片包含资源名称、提供方、适用条件、联系方式和申请入口。用户可以收藏感兴趣的资源，并跟踪申请状态。

4.4.4 团队协作面板

团队协作面板支持团队成员管理和协作沟通。面板包含团队成员列表（显示成员头像、姓名、角色、在线状态）、任务分配视图（展示每个成员的任务负载和完成情况）和协作消息流（展示团队成员的动态通知和讨论记录）。

团队协作面板支持角色权限管理，团队创建者可以邀请成员、分配角色（管理员、核心成员、普通成员）和设置权限。面板还支持团队公告发布和任务@提醒功能。

4.5 AI对话服务设计

AI对话服务是支撑四大AI智能体的核心服务层，负责与外部大语言模型API通信，实现多轮对话、工具调用和流式响应等功能。

4.5.1 多轮对话机制

AI对话服务支持多轮对话，即在一次会话中，AI能够理解上下文并根据历史消息生成连贯的回复。多轮对话的实现机制包括：消息历史管理（维护完整的对话消息列表）、上下文窗口控制（当消息数量超过上下文窗口限制时，自动截断或压缩早期消息）、系统提示词注入（在每次请求中注入AI角色的系统提示词，确保AI保持角色一致性）。

4.5.2 工具调用（Tool Calling）

AI对话服务支持Tool Calling功能，允许AI在对话过程中动态调用外部工具函数。工具调用的流程为：第一步，AI根据用户意图判断是否需要调用工具；第二步，如果需要，AI返回工具调用请求（包含工具名称和参数）；第三步，前端执行工具函数并返回结果；第四步，AI根据工具结果生成最终回复。

```
// 工具调用数据结构
interface ToolCall {
  id: string;
  toolName: string;
  parameters: Record<string, any>;
  result?: any;
  status: 'pending' | 'success' | 'error';
}

// 工具定义
interface ToolDefinition {
  name: string;
  description: string;
  parameters: {
    type: string;
    properties: Record<string, { type: string; description: string }>;
    required: string[];
  };
}

// 平台注册的工具体型
const registeredTools: ToolDefinition[] = [
  {
    name: 'search_market_data',
    description: '搜索指定行业的市场数据',
    parameters: {
      type: 'object',
      properties: {
        industry: { type: 'string', description: '行业名称' },
        region: { type: 'string', description: '地区' },
      },
      required: ['industry'],
    },
  },
]
```

```
    },
  },
  {
    name: 'generate_bmc',
    description: '生成商业模式画布',
    parameters: {
      type: 'object',
      properties: {
        projectName: { type: 'string', description: '项目名称' },
        description: { type: 'string', description: '项目描述' },
      },
      required: ['projectName', 'description'],
    },
  },
],
```

4.5.3 流式响应处理

AI对话服务使用流式响应（Streaming Response）技术，使AI的回复能够逐字逐句实时显示，提升用户体验。流式响应的实现基于Server-Sent Events（SSE）或Fetch Streaming API，前端通过ReadableStream读取响应体，逐块解析并更新消息内容。

```
// 流式响应处理
async function streamChatResponse(
  messages: Message[],
  onChunk: (chunk: string) => void,
  onDone: () => void,
  onError: (error: Error) => void
) {
  try {
    const response = await fetch('/api/ai/chat/stream', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ messages, stream: true }),
    });

    const reader = response.body?.getReader();
    const decoder = new TextDecoder();
    let buffer = '';

    while (true) {
      const { done, value } = await reader.read();
      if (done) { onDone(); break; }

      buffer += decoder.decode(value, { stream: true });
      const lines = buffer.split('\n');
      buffer = lines.pop() || '';

      for (const line of lines) {
        if (line.startsWith('data: ')) {
```

```
        const data = line.slice(6);
        if (data === '[DONE]') { onDone(); return; }
        const parsed = JSON.parse(data);
        if (parsed.choices?.[0]?.delta?.content) {
            onChunk(parsed.choices[0].delta.content);
        }
    }
}
}
} catch (error) {
    onError(error as Error);
}
}
```

4.5.4 消息结构设计

AI对话服务使用统一的消息结构（Message）和会话结构（Conversation）来管理对话数据。

```
// 消息结构
interface Message {
    id: string;
    role: 'system' | 'user' | 'assistant' | 'tool';
    content: string;
    timestamp: number;
    model?: string;
    toolCalls?: ToolCall[];
    metadata?: {
        tokens?: { prompt: number; completion: number };
        finishReason?: string;
    };
};

// 会话结构
interface Conversation {
    id: string;
    title: string;
    role: AIRole;
    messages: Message[];
    createdAt: number;
    updatedAt: number;
    modelConfig: ModelConfig;
}
```

Message结构支持四种角色类型：system（系统提示词，定义AI角色的行为规范）、user（用户消息）、assistant（AI回复）和tool（工具调用结果）。每条消息包含唯一ID、角色类型、内容文本、时间戳和可选的元数据（Token消耗量、模型名称等）。

Conversation结构管理一次完整的对话会话，包含会话ID、标题、关联的AI角色、消息列表、创建时间和更新时间。一个用户可以创建多个会话，每个会话独立维护消息历史和上下文。

5 工具箱模块设计

工具箱模块是青宸智汇平台的能力底座，为四大AI智能体提供通用的工具和资源管理功能。工具箱包含Skill库管理、MCP协议集成、自动化触发引擎、知识库检索、应用中心、用量统计、消息中心和记忆管理等八个功能子模块。本章详细描述各子模块的设计方案。

5.1 Skill库管理

Skill库是平台内置的技能管理模块，包含一系列预设的AI技能模板。每个Skill定义了特定场景下的AI交互流程、系统提示词模板、输入参数规范和输出格式约束。用户可以浏览、启用、配置和管理这些技能，快速获得针对性的AI服务。

5.1.1 Skill数据结构

```
interface Skill {
  id: string;
  name: string;
  description: string;
  category: SkillCategory;
  icon: string;
  systemPrompt: string;
  inputSchema: {
    type: 'object';
    properties: Record<string, SkillParam>;
    required: string[];
  };
  outputFormat: 'text' | 'json' | 'markdown' | 'chart';
  tags: string[];
  enabled: boolean;
  usageCount: number;
  rating: number;
  author: string;
  version: string;
  createdAt: string;
  updatedAt: string;
}

type SkillCategory =
  | 'market_analysis' // 市场分析
  | 'strategy'         // 战略规划
  | 'product'          // 产品制作
  | 'management'       // 项目管理
  | 'document'         // 文档撰写
  | 'data_viz'         // 数据可视化
  | 'custom';          // 自定义

interface SkillParam {
  type: 'string' | 'number' | 'boolean' | 'select';
  label: string;
  description?: string;
  options?: { label: string; value: string }[];
  default?: any;
  required?: boolean;
}
```

5.1.2 Skill库功能设计

Skill库管理模块提供以下核心功能：

技能浏览——以卡片网格形式展示所有可用技能，支持按分类筛选、按标签搜索和按使用频率排序。每个技能卡片显示技能名称、图标、描述、使用次数和评分。用户点击卡片查看技能详情和参数配置界面。

技能启用与配置——用户可以启用或禁用技能。启用后的技能会出现在AI对话的工具列表中，AI可根据用户意图自动调用。技能配置允许用户设置输入参数的默认值和约束条件。

技能创建——高级用户可以创建自定义技能。创建界面提供系统提示词编辑器、输入参数Schema定义器和输出格式选择器。用户可以测试技能效果并发布到个人技能库。

技能版本管理——每个技能支持版本管理，记录每次修改的历史版本。用户可以回退到任意历史版本，也可以查看版本变更日志。

技能分享与社区——用户可以将自定义技能分享到社区，其他用户可以浏览、评分和使用社区分享的技能。技能社区形成技能生态，促进技能的复用和优化。

5.2 MCP协议集成

MCP (Model Context Protocol) 是平台用于集成外部工具服务的标准化协议。通过MCP，AI智能体可以动态连接外部数据源和工具服务，扩展AI的能力边界。

5.2.1 MCP架构设计

MCP集成模块的架构包含三个层次：MCP客户端层、MCP服务注册层和MCP服务执行层。

MCP客户端层负责与MCP服务端建立连接，支持标准MCP协议的消息格式（JSON-RPC 2.0）。客户端层管理连接的生命周期，包括连接建立、心跳保活、断线重连和连接关闭。

MCP服务注册层管理所有已注册的MCP服务。每个MCP服务定义包含服务名称、服务描述、服务端点URL、认证方式和工具列表。注册层提供服务发现、服务健康检查和服务状态监控功能。

MCP服务执行层负责实际调用MCP服务的工具函数。当AI需要调用外部工具时，执行层根据工具名称查找对应的MCP服务，构建调用请求（包含工具名称和参数），发送到MCP服务端，并解析返回结果。

5.2.2 MCP配置管理

```
// MCP服务配置结构
interface MCPServiceConfig {
    id: string;
    name: string;
    description: string;
    endpoint: string;
    protocol: 'http' | 'websocket' | 'stdio';
    auth: {
        type: 'none' | 'bearer' | 'apikey';
        token?: string;
        headerName?: string;
    };
    tools: MCPTool[];
    status: 'connected' | 'disconnected' | 'error';
    lastHealthCheck?: number;
}

interface MCPTool {
    name: string;
    description: string;
    inputSchema: JSONSchema;
    outputSchema?: JSONSchema;
}
```

MCP配置管理界面提供以下功能：服务添加（填写服务名称、端点URL、认证信息等）、服务编辑（修改服务配置）、服务删除（移除不再使用的服务）、服务测试（发送测试请求验证连通性）和工具浏览（查看服务提供的工具列表和参数定义）。

5.3 自动化触发引擎

自动化触发引擎是工具箱的智能调度模块，支持基于条件的事件触发和任务自动执行。用户可以定义自动化规则，当满足触发条件时，自动执行预设的动作序列。

5.3.1 自动化规则数据结构

```
interface AutomationRule {
    id: string;
    name: string;
    description: string;
    enabled: boolean;
    trigger: AutomationTrigger;
    conditions: AutomationCondition[];
    actions: AutomationAction[];
    executionHistory: AutomationExecution[];
}

interface AutomationTrigger {
    type: 'schedule' // 定时触发
    | 'event' // 事件触发
    | 'webhook' // Webhook触发
    | 'manual'; // 手动触发
    config: {
        cronExpression?: string; // 定时表达式
        eventType?: string; // 事件类型
        webhookUrl?: string; // Webhook URL
    };
}

interface AutomationCondition {
    field: string;
    operator: 'eq' | 'ne' | 'gt' | 'lt' | 'contains' | 'in';
    value: any;
}

interface AutomationAction {
    type: 'call_ai' // 调用AI
    | 'call_tool' // 调用工具
    | 'send_message' // 发送消息
    | 'update_data' // 更新数据
    | 'send_notify'; // 发送通知
    params: Record<string, any>;
}
```

5.3.2 自动化执行流程

自动化触发引擎的执行流程包括：触发检测（监听定时器、事件或Webhook，检测是否满足触发条件）、条件评估（检查所有条件是否满足）、动作执行（依次执行预设的动作序列）和结果记录（记录执行日志和执行结果）。

自动化引擎支持错误处理和重试机制。当某个动作执行失败时，引擎根据配置选择跳过、重试或终止整个规则执行。执行日志详细记录每一步的执行时间、执行结果和错误信息，方便用户排查问题。

5.4 知识库检索

知识库检索模块为AI智能体提供知识检索和语义搜索能力。知识库包含行业知识、创业方法论、案例分析、政策法规等内容，支持全文检索和语义相似度搜索。

5.4.1 知识库架构

知识库模块采用向量数据库+全文索引的混合检索架构。文档入库时，首先进行文本预处理（分词、清洗、去重），然后将文档切分为语义段落（Chunk），对每个段落生成向量嵌入（Embedding），存入向量数据库。同时建立全文倒排索引，支持关键词精确匹配。

检索时，系统同时执行向量相似度搜索和全文检索，将两种检索结果融合排序，返回最相关的知识片段。这种混合检索方式兼顾了语义相似性和关键词精确匹配，提高检索的准确率和召回率。

5.4.2 知识库管理功能

知识库管理界面提供以下功能：文档上传（支持PDF、Word、Markdown、TXT等格式）、文档分类（按行业、主题等分类管理）、文档预处理（自动分词、清洗、分块）、索引管理（查看和重建索引）、检索测试（输入查询测试检索效果）和知识图谱（可视化展示知识关联关系）。

5.5 用量统计与Token监控

用量统计模块实时监控AI调用的Token消耗量和API使用情况，帮助用户了解和管理API使用成本。

5.5.1 统计数据结构

```
// 用量统计数据结构
interface UsageStats {
    totalTokens: number;
    promptTokens: number;
    completionTokens: number;
    totalRequests: number;
    totalCost: number;
    dailyStats: DailyUsage[];
    modelStats: ModelUsage[];
    roleStats: RoleUsage[];
}

interface DailyUsage {
    date: string;
    tokens: number;
    requests: number;
    cost: number;
}

interface ModelUsage {
    model: string;
    tokens: number;
    requests: number;
    cost: number;
}

interface RoleUsage {
    role: AIRole;
    tokens: number;
    requests: number;
    avgResponseTime: number;
}
```

5.5.2 统计展示设计

用量统计模块的展示界面包含以下内容：

总览面板——以数字卡片形式展示总Token消耗、总API请求数、总成本和平均响应时间。每个数字卡片配有环比增长率指示器，直观反映用量变化趋势。

趋势图——以面积图形式展示每日Token消耗趋势，支持按天、周、月切换时间粒度。趋势图下方叠加请求次数曲线，方便对比Token消耗与请求频次的关系。

模型分布——以环形图形式展示各AI模型的Token消耗占比，帮助用户了解不同模型的使用分布。

角色分布——以柱状图形式展示各AI角色的Token消耗量，帮助用户了解哪个AI角色消耗最多资源。

明细表格——以表格形式展示详细的API调用记录，包括调用时间、AI角色、模型名称、输入Token、输出Token、成本和耗时。表格支持分页、排序和导出。

5.6 应用中心

应用中心模块是平台的应用商店，展示和管理可用的应用插件和扩展功能。应用中心包含官方应用和社区应用两类，用户可以浏览、安装和管理应用。

应用中心的应用类型包括：数据源应用（接入外部数据源，如行业数据库、政策库）、工具应用（提供特定功能工具，如财务计算器、SWOT分析器）、模板应用（提供预设的项目模板和文档模板）和集成应用（与第三方平台集成，如GitHub、Figma等）。

5.7 消息中心

消息中心模块统一管理系统通知和AI消息。消息类型包括：系统通知（版本更新、维护公告等）、任务提醒（任务截止日期提醒、自动化规则执行通知等）、AI消息（AI分析完成通知、工具调用结果等）和协作消息（团队成员动态、@提醒等）。

消息中心提供消息列表视图（按时间倒序展示所有消息）、消息分类筛选（按消息类型筛选）、消息已读/未读管理（标记消息已读状态）、消息批量操作（批量已读、批量删除）和消息详情查看等功能。

5.8 记忆管理

记忆管理模块维护AI对话的上下文和用户偏好。记忆系统分为短期记忆和长期记忆两个层次。

短期记忆存储当前会话的上下文信息，包括最近的消息历史、当前话题和用户意图。短期记忆在会话结束或超时后自动清除。

长期记忆存储用户的偏好设置和重要信息，包括用户的基本信息（姓名、学校、专业等）、创业项目信息（项目名称、行业、阶段等）、交互偏好（喜欢的回答风格、常用功能等）和重要决策记录。长期记忆持久化到本地存储，在后续会话中自动加载。

```
// 记忆数据结构
interface Memory {
  id: string;
  userId: string;
  type: 'short_term' | 'long_term';
  category: 'user_info' | 'project_info' | 'preference' | 'decision' | 'context';
  key: string;
  value: any;
  importance: number; // 0-1
  createdAt: number;
  lastAccessedAt: number;
  accessCount: number;
  expiresAt?: number; // 短期记忆的过期时间
}
```

6 数据结构设计

本章详细描述青宸智汇平台的核心数据结构设计，包括核心类型定义、状态数据结构、思维导图数据结构和 服务层数据接口。数据结构的设计遵循类型安全、语义清晰和易于序列化的原则。

6.1 核心类型定义

6.1.1 AIRole类型

AIRole类型定义了平台的四个AI智能体角色枚举。

```
type AIRole =  
  | 'explorer'      // 探路者AI - 市场分析  
  | 'strategist'    // 军师AI - 战略规划  
  | 'artisan'       // 工匠AI - 产品制作  
  | 'steward';      // 管家AI - 项目管理  
  
interface AIRoleConfig {  
  role: AIRole;  
  name: string;  
  nameEn: string;  
  icon: string;  
  color: string;  
  description: string;  
  systemPrompt: string;  
  panels: AppPage[];  
  defaultModel: string;  
  tools: string[];  
}
```

6.1.2 AppPage类型

AppPage类型定义了AI角色下的功能面板配置。

```
interface AppPage {  
  id: string;  
  title: string;  
  icon: string;  
  component: React.ComponentType;  
  order: number;  
  visible: boolean;  
  badge?: number;  
  description?: string;  
}
```

6.1.3 Message类型

Message类型定义了AI对话中的消息结构，支持四种角色类型和工具调用。

```
interface Message {  
  id: string;  
  role: 'system' | 'user' | 'assistant' | 'tool';  
}
```

```
content: string;
timestamp: number;
model?: string;
toolCalls?: ToolCall[];
status: 'sending' | 'streaming' | 'complete' | 'error';
error?: string;
metadata?: {
  tokens?: { prompt: number; completion: number };
  latency?: number;
  finishReason?: string;
};
}
```

6.1.4 Conversation类型

Conversation类型管理一次完整的AI对话会话。

```
interface Conversation {
  id: string;
  title: string;
  role: AIRole;
  messages: Message[];
  createdAt: number;
  updatedAt: number;
  modelConfig: ModelConfig;
  pinned: boolean;
  summary?: string;
  tags?: string[];
}
```

6.1.5 ModelConfig类型

ModelConfig类型定义了AI模型的配置参数。

```
interface ModelConfig {
  model: string;
  temperature: number; // 0-2, 默认0.7
  maxTokens: number; // 最大输出Token数
  topP: number; // 核采样概率, 默认1
  frequencyPenalty: number; // 频率惩罚, 默认0
  presencePenalty: number; // 存在惩罚, 默认0
  stream: boolean; // 是否流式输出
  tools?: ToolDefinition[]; // 可用工具列表
}
```

6.2 状态数据结构

状态数据结构定义了Zustand各Store管理的状态形状。以下描述主要Store的状态结构。

6.2.1 认证状态结构

```
interface AuthState {
  user: User | null;
  token: string | null;
  isAuthenticated: boolean;
  loading: boolean;
  error: string | null;

  login: (credentials: LoginCredentials) => Promise<void>;
  logout: () => void;
  restoreSession: () => Promise<void>;
  updateUser: (updates: Partial<User>) => void;
}

interface User {
  id: string;
  username: string;
  email: string;
  avatar: string;
  role: 'student' | 'mentor' | 'admin';
  school?: string;
  major?: string;
  createdAt: number;
}
```

6.2.2 AI对话状态结构

```
interface AIChatState {
  conversations: Conversation[];
  currentConversationId: string | null;
  currentMessages: Message[];
  isStreaming: boolean;
  modelConfig: ModelConfig;
  currentRole: AIRole;

  sendMessage: (content: string) => Promise<void>;
  receiveStreamChunk: (chunk: string) => void;
  createConversation: (role: AIRole) => void;
  deleteConversation: (id: string) => void;
  switchConversation: (id: string) => void;
  setModelConfig: (config: Partial<ModelConfig>) => void;
  setRole: (role: AIRole) => void;
  clearMessages: () => void;
}
```

6.2.3 应用全局状态结构

```
interface AppState {  
  sidebarCollapsed: boolean;  
  currentRole: ARole;  
  currentPage: string;  
  theme: 'light' | 'dark';  
  locale: 'zh-CN' | 'en-US';  
  
  toggleSidebar: () => void;  
  setRole: (role: ARole) => void;  
  setCurrentPage: (page: string) => void;  
  setTheme: (theme: 'light' | 'dark') => void;  
}
```

6.3 思维导图数据结构

思维导图编辑器是平台的自研特色组件，其数据结构设计支持树形层级、多布局模式和丰富的节点属性。

6.3.1 MindMapNode类型

```
interface MindMapNode {
  id: string;
  parentId: string | null;
  text: string;
  note?: string;
  level: number;
  order: number;
  collapsed: boolean;
  style: NodeStyle;
  children: MindMapNode[];

  // 布局计算结果（由布局算法填充）
  layout?: {
    x: number;
    y: number;
    width: number;
    height: number;
  };
}

interface NodeStyle {
  fontSize: number;
  fontWeight: 'normal' | 'bold';
  color: string;
  backgroundColor: string;
  borderColor: string;
  borderWidth: number;
  borderRadius: number;
  padding: number;
  shape: 'rect' | 'rounded' | 'ellipse' | 'diamond';
}
```

6.3.2 MindMapData类型

```
interface MindMapData {
  id: string;
  title: string;
  root: MindMapNode;
  layoutMode: LayoutMode;
  theme: MindMapTheme;
  createdAt: number;
  updatedAt: number;
  metadata: {
    nodeCount: number;
    maxDepth: number;
    tags: string[];
  };
}
```

6.3.3 LayoutMode类型

```
type LayoutMode =
  | 'tree-right'      // 树形布局（从左到右）
  | 'tree-left'       // 树形布局（从右到左）
  | 'tree-top'        // 树形布局（从上到下）
  | 'tree-bottom'     // 树形布局（从下到上）
  | 'radial'          // 放射布局
  | 'fishbone'        // 鱼骨图布局
  | 'outline';        // 大纲模式（列表布局）

// 布局算法接口
interface LayoutEngine {
  layout(root: MindMapNode, mode: LayoutMode, options: LayoutOptions): void;
  getNodeBounds(node: MindMapNode): Bounds;
  getConnections(root: MindMapNode): Connection[];
}
```

自研树形布局算法是思维导图编辑器的核心技术。算法采用递归布局策略：首先计算每个节点的子树高度（后序遍历），然后根据子树高度计算节点的垂直位置（前序遍历），最后根据层级计算节点的水平位置。算法支持节点折叠、自定义间距和多种方向布局。

6.4 服务层数据接口

服务层数据接口定义了前端与后端API通信的数据契约。以下描述主要服务的数据接口。

6.4.1 AI服务接口数据

```
// AI对话请求
interface ChatRequest {
    messages: Message[];
    model: string;
    temperature: number;
    maxTokens: number;
    stream: boolean;
    tools?: ToolDefinition[];
}

// AI对话响应
interface ChatResponse {
    id: string;
    model: string;
    choices: {
        message: Message;
        finishReason: string;
        index: number;
    }[];
    usage: {
        promptTokens: number;
        completionTokens: number;
        totalTokens: number;
    };
}

// 流式响应块
interface StreamChunk {
    id: string;
    choices: {
        delta: {
            content?: string;
            role?: string;
            toolCalls?: ToolCall[];
        };
        finishReason?: string;
    }[];
}
```

6.4.2 认证服务接口数据

```
// 登录请求
interface LoginRequest {
    username: string;
    password: string;
    captcha?: string;
}

// 登录响应
interface LoginResponse {
    token: string;
    refreshToken: string;
    expiresIn: number;
    user: User;
}

// 注册请求
interface RegisterRequest {
    username: string;
    email: string;
    password: string;
    school?: string;
    major?: string;
    verificationCode: string;
}

// 会话恢复响应
interface SessionRestoreResponse {
    valid: boolean;
    user: User | null;
    token: string | null;
}
```

6.4.3 知识服务接口数据

```
// 知识检索请求
interface KnowledgeSearchRequest {
    query: string;
    category?: string;
    topK?: number;
    threshold?: number;
}

// 知识检索结果
interface KnowledgeSearchResult {
    id: string;
    title: string;
    content: string;
    score: number;
    source: string;
    metadata: {
        category: string;
        tags: string[];
        publishedAt?: number;
    };
};
}
```

6.4.4 行业数据服务接口数据

```
// 行业报告请求
interface IndustryReportRequest {
    industry: string;
    region?: string;
    startDate?: string;
    endDate?: string;
}

// 行业数据响应
interface IndustryDataResponse {
    industry: string;
    marketSize: number;
    growthRate: number;
    trends: TrendData[];
    competitors: CompetitorData[];
    policies: PolicyData[];
}
}
```

7 接口设计

本章描述青宸智汇平台的接口设计方案，包括AI服务接口、认证服务接口、知识服务接口、HTTP请求封装和行业数据服务接口。所有接口采用RESTful风格，数据格式为JSON。

7.1 AI服务接口

7.1.1 对话接口

AI对话接口是平台最核心的接口，负责将用户的对话消息发送至大语言模型API，并返回AI生成的回复。接口支持普通响应和流式响应两种模式。

```
// POST /api/ai/chat
// 请求体
{
  "messages": [
    { "role": "system", "content": "你是一个市场分析专家..." },
    { "role": "user", "content": "分析一下人工智能教育行业" }
  ],
  "model": "gpt-4o",
  "temperature": 0.7,
  "maxTokens": 4096,
  "stream": false,
  "tools": [...]
}

// 响应体
{
  "id": "chatcmpl-xxx",
  "model": "gpt-4o",
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "人工智能教育行业是一个..."
    },
    "finishReason": "stop"
  }],
  "usage": {
    "promptTokens": 156,
    "completionTokens": 892,
    "totalTokens": 1048
  }
}
```

流式响应接口（POST /api/ai/chat/stream）使用Server-Sent Events格式，响应体为一系列data:前缀的JSON块，每块包含部分回复内容。流式接口使AI回复能够实时逐字显示，显著提升用户体验。

7.1.2 模型配置接口

```
// GET /api/ai/models - 获取可用模型列表
// 响应
{
  "models": [
    {
      "id": "gpt-4o",
      "name": "GPT-4o",
      "description": "最新多模态模型",
      "contextWindow": 128000,
      "maxOutput": 16384,
      "pricing": { "input": 0.005, "output": 0.015 }
    },
    {
      "id": "gpt-4o-mini",
      "name": "GPT-4o Mini",
      "description": "轻量快速模型",
      "contextWindow": 128000,
      "maxOutput": 16384,
      "pricing": { "input": 0.00015, "output": 0.0006 }
    }
  ]
}

// PUT /api/ai/model-config - 更新模型配置
// 请求体
{
  "model": "gpt-4o",
  "temperature": 0.7,
  "maxTokens": 4096
}
```

7.2 认证服务接口

7.2.1 登录接口

```
// POST /api/auth/login
// 请求体
{
  "username": "student001",
  "password": "encrypted_password",
  "captcha": "abcd"
}

// 响应体 (200)
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "refreshToken": "refresh_token_xxx",
  "expiresIn": 7200,
  "user": {
    "id": "u_001",
    "username": "student001",
    "email": "student@university.edu",
    "avatar": "/avatars/u_001.png",
    "role": "student",
    "school": "XX大学",
    "major": "计算机科学"
  }
}

// 错误响应 (401)
{
  "error": "invalid_credentials",
  "message": "用户名或密码错误"
}
```

7.2.2 注册接口

```
// POST /api/auth/register
// 请求体
{
  "username": "newstudent",
  "email": "new@university.edu",
  "password": "encrypted_password",
  "school": "XX大学",
  "major": "软件工程",
  "verificationCode": "123456"
}

// 响应体 (201)
{
  "userId": "u_002",
  "username": "newstudent",
  "message": "注册成功, 请登录"
}
```

7.2.3 会话恢复接口

```
// POST /api/auth/session
// 请求头
// Authorization: Bearer <token>

// 响应体 (200)
{
  "valid": true,
  "user": {
    "id": "u_001",
    "username": "student001",
    "email": "student@university.edu",
    "role": "student"
  },
  "token": "new_token_if_refreshed"
}

// 响应体 (401) - Token过期
{
  "valid": false,
  "error": "token_expired",
  "message": "会话已过期, 请重新登录"
}
```

7.2.4 Token刷新接口

```
// POST /api/auth/refresh
// 请求体
{
  "refreshToken": "refresh_token_xxx"
}

// 响应体 (200)
{
  "token": "new_access_token",
  "refreshToken": "new_refresh_token",
  "expiresIn": 7200
}
```

7.3 知识服务接口

```
// POST /api/knowledge/search
// 请求体
{
  "query": "人工智能行业市场规模",
  "category": "industry_report",
  "topK": 5,
  "threshold": 0.7
}

// 响应体
{
  "results": [
    {
      "id": "doc_001",
      "title": "2024年中国人工智能行业报告",
      "content": "中国人工智能市场规模在2024年达到...",
      "score": 0.92,
      "source": "艾瑞咨询",
      "metadata": {
        "category": "industry_report",
        "tags": ["AI", "市场规模"],
        "publishedAt": 1704067200
      }
    }
  ],
  "total": 15
}
```

知识服务还提供文档管理接口：POST `/api/knowledge/documents`（上传文档）、GET `/api/knowledge/documents`（获取文档列表）、DELETE `/api/knowledge/documents/:id`（删除文档）和POST `/api/knowledge/index/rebuild`（重建索引）。

7.4 HTTP请求封装

平台的HTTP请求封装基于Fetch API，提供统一的请求拦截器、响应拦截器和错误处理机制。

7.4.1 请求拦截器

```
// HTTP请求封装
class HttpClient {
  private baseUrl: string;
  private interceptors: Interceptor[];

  async request<T>(config: RequestConfig): Promise<T> {
    // 请求拦截
    for (const interceptor of this.interceptors) {
      if (interceptor.request) {
        config = await interceptor.request(config);
      }
    }

    // 添加认证Token
    const token = useAuthStore.getState().token;
    if (token) {
      config.headers = {
        ...config.headers,
        'Authorization': `Bearer ${token}`,
      };
    }

    // 发送请求
    const response = await fetch(this.baseUrl + config.url, {
      method: config.method || 'GET',
      headers: { 'Content-Type': 'application/json', ...config.headers },
      body: config.data ? JSON.stringify(config.data) : undefined,
      signal: config.signal,
    });

    // 响应拦截
    let result = response;
    for (const interceptor of this.interceptors) {
      if (interceptor.response) {
        result = await interceptor.response(result);
      }
    }

    if (!result.ok) {
      throw await this.handleError(result);
    }

    return result.json();
  }
}
```

7.4.2 错误处理

```
// 错误处理
async function handleError(response: Response): Promise<ApiError> {
  let errorBody: any;
  try {
    errorBody = await response.json();
  } catch {
    errorBody = { message: response.statusText };
  }

  switch (response.status) {
    case 401:
      // Token过期, 尝试刷新
      if (!isRefreshing) {
        isRefreshing = true;
        const refreshed = await refreshToken();
        isRefreshing = false;
        if (refreshed) {
          return retryRequest(response.url);
        }
      }
      // 刷新失败, 跳转登录
      useAuthStore.getState().logout();
      redirectToLogin();
      return { code: 401, message: '登录已过期, 请重新登录' };

    case 403:
      return { code: 403, message: '没有权限执行此操作' };

    case 429:
      return { code: 429, message: '请求过于频繁, 请稍后再试' };

    case 500:
      return { code: 500, message: '服务器内部错误, 请稍后重试' };

    default:
      return {
        code: response.status,
        message: errorBody.message || '请求失败',
      };
  }
}
```

7.5 行业数据服务接口

```
// GET /api/industry/reports?industry=AI&region=china
// 响应体
{
  "reports": [
    {
      "id": "rpt_001",
      "title": "2024中国人工智能产业发展白皮书",
      "industry": "AI",
      "region": "china",
      "publishDate": "2024-03-15",
      "source": "中国信通院",
      "summary": "本报告全面分析了...",
      "downloadUrl": "/files/rpt_001.pdf",
      "pages": 128
    }
  ],
  "total": 42,
  "page": 1,
  "pageSize": 20
}

// GET /api/industry/data?industry=AI&metric=market_size&years=2020-2024
// 响应体
{
  "industry": "AI",
  "metric": "market_size",
  "unit": "亿元",
  "data": [
    { "year": 2020, "value": 1280 },
    { "year": 2021, "value": 1860 },
    { "year": 2022, "value": 2580 },
    { "year": 2023, "value": 3770 },
    { "year": 2024, "value": 5780 }
  ],
  "growthRates": [0.45, 0.39, 0.46, 0.53],
  "forecast": [
    { "year": 2025, "value": 8200, "confidence": 0.85 }
  ]
}
```

行业数据服务还提供政策查询接口（GET /api/industry/policies）和竞品数据接口（GET /api/industry/competitors），返回结构化的政策法规列表和竞争对手数据。

8 界面设计

本章描述青宸智汇平台的界面设计方案，包括整体布局、AI角色多面板布局、登录页面、3D仪表盘和数据可视化SVG组件库设计。

8.1 整体布局

平台采用经典的“侧边栏+主内容区”布局结构。侧边栏宽度200px，固定在页面左侧，包含Logo区域、导航菜单和用户信息区域。主内容区占据剩余空间，包含顶部工具栏和内容区域。

8.1.1 侧边栏设计

侧边栏从上到下依次为：Logo区域（显示“青宸智汇”品牌Logo和名称）、AI角色导航区域（四个AI角色的切换按钮，每个按钮显示角色图标和名称）、功能菜单区域（工具箱、数据服务、思维导图等功能入口）和用户信息区域（显示当前用户头像、名称和下拉菜单）。

侧边栏支持折叠/展开切换。折叠状态下，侧边栏宽度缩小为64px，仅显示图标。展开状态下显示完整的图标和文字。折叠状态通过Zustand的全局状态管理，在页面刷新后保持。

8.1.2 主内容区设计

主内容区包含顶部工具栏和内容区域。顶部工具栏高度48px，包含面包屑导航（显示当前页面路径）、搜索框（全局搜索功能）、通知图标（显示未读消息数量徽标）和用户头像下拉菜单。内容区域根据当前激活的功能模块动态渲染对应的页面组件。

8.2 AI角色多面板布局（AIRoleLayout组件）

AIRoleLayout组件是AI智能体模块的统一布局容器，负责管理AI角色下多个功能面板的切换和展示。组件设计如下：

```
// AIRoleLayout组件结构
interface AIRoleLayoutProps {
  role: AIRole;
  panels: AppPage[];
  children: React.ReactNode;
}

const AIRoleLayout: React.FC<AIRoleLayoutProps> = ({ role, panels, children }) => {
  const [activePanel, setActivePanel] = useState(panels[0]?.id);

  return (
    <div className="ai-role-layout">
      { /* AI角色头部区域 */ }
      <div className="role-header">
        <RoleAvatar role={role} size="large" />
        <RoleInfo role={role} />
        <RoleActions role={role} />
      </div>
    </div>
  );
}
```

```
    {/* 面板标签栏 */}
    <div className="panel-tabs">
      {panels.map(panel => (
        <PanelTab
          key={panel.id}
          panel={panel}
          active={activePanel === panel.id}
          onClick={() => setActivePanel(panel.id)}
        />
      ))}
    </div>

    {/* 面板内容区域 */}
    <div className="panel-content">
      {panels.map(panel => (
        <PanelRenderer
          key={panel.id}
          panel={panel}
          visible={activePanel === panel.id}
        />
      ))}
    </div>

    {/* AI对话区域 */}
    <div className="chat-area">
      <ChatPanel role={role} />
    </div>
  </div>
);
};
```

AIRoleLayout组件采用“头部+标签栏+内容区+对话区”的四区域布局。头部区域显示AI角色的头像、名称、简介和操作按钮（开始对话、查看历史等）。标签栏以水平标签形式展示该AI角色下的所有功能面板，支持标签切换和拖拽排序。内容区域根据当前激活的标签渲染对应的面板组件。对话区域固定在底部或侧边，提供AI对话交互界面。

8.2.1 面板切换动画

面板切换时使用CSS Transition实现平滑过渡动画。切换动画包含两个阶段：退出动画（当前面板淡出，向下位移）和进入动画（新面板淡入，从下方位移进入）。动画时长300ms，使用ease-out缓动函数。动画过程中内容区域不会闪烁，提供流畅的视觉体验。

8.2.2 对话面板设计

AI对话面板是AIRoleLayout的核心交互区域。对话面板包含消息列表区域和输入区域。消息列表区域以气泡形式展示对话消息，用户消息右对齐，AI消息左对齐。AI消息支持Markdown格式渲染，包括标题、列表、代码块、表格等。流式响应时，消息内容实时追加，末尾显示打字光标动画。

输入区域包含多行文本输入框、发送按钮、附件上传按钮和模型配置按钮。输入框支持快捷键（Ctrl+Enter发送、Shift+Enter换行）。发送按钮在AI回复流式输出期间变为“停止生成”按钮，允许用户中断AI回复。

8.3 登录页面设计

登录页面是用户进入平台的第一界面，设计上注重简洁美观和品牌展示。页面采用全屏布局，左侧为品牌展示区（占60%宽度），右侧为登录表单区（占40%宽度）。

品牌展示区使用渐变背景色（从深蓝到浅蓝），中央显示“青宸智汇”品牌Logo和标语“大学生创业智能体平台”。背景包含动态粒子动画效果，增强视觉吸引力。品牌展示区还展示平台的四个AI角色图标和简短描述，让用户快速了解平台功能。

登录表单区使用白色背景，中央放置登录表单卡片。表单包含用户名输入框、密码输入框、验证码输入框、记住密码复选框和登录按钮。表单下方提供“忘记密码”链接和“注册账号”链接。表单采用Ant Design的Form组件实现，内置表单验证（用户名非空、密码长度6位以上、验证码正确）。

```
// 登录页面组件结构
const LoginPage: React.FC = () => {
  const [form] = Form.useForm();
  const { login, loading } = useAuthStore();

  const handleSubmit = async (values: LoginRequest) => {
    try {
      await login(values);
      message.success('登录成功');
      navigate('/app');
    } catch (error) {
      message.error(error.message || '登录失败');
    }
  };

  return (
    <div className="login-page">
      <div className="brand-section">
        <BrandLogo size="large" />
        <h1>青宸智汇</h1>
        <p>大学生创业智能体平台</p>
        <AIRolePreview />
      </div>
      <div className="form-section">
        <Card className="login-card">
          <h2>欢迎登录</h2>
          <Form form={form} onFinish={handleSubmit}>
            <Form.Item name="username" rules={[{required: true}]}>
              <Input prefix={<UserOutlined />} placeholder="用户名" />
            </Form.Item>
            <Form.Item name="password" rules={[{required: true, min: 6}]}>
              <Input.Password prefix={<LockOutlined />} placeholder="密码" />
            </Form.Item>
          </Form>
        </Card>
      </div>
    </div>
  );
};
```

```
        </Form.Item>
        <Form.Item name="captcha">
          <CaptchaInput />
        </Form.Item>
        <Form.Item>
          <Button type="primary" htmlType="submit" block loading={loading}>
            登录
          </Button>
        </Form.Item>
      </Form>
    </Card>
  </div>
</div>
);
};
```

8.4 3D仪表盘（Dashboard3D）

3D仪表盘是平台的特色可视化组件，使用Three.js实现3D数据展示效果。Dashboard3D组件将关键运营指标以3D形式呈现，包括3D球体（展示总体Token消耗）、立体柱状图（展示各AI角色用量）、3D环形图（展示模型分布）和粒子效果（展示实时活动状态）。

8.4.1 3D场景设计

```
// Dashboard3D组件核心结构
const Dashboard3D: React.FC<{ data: UsageStats }> = ({ data }) => {
  const mountRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    if (!mountRef.current) return;

    // 创建场景
    const scene = new THREE.Scene();
    scene.fog = new THREE.Fog(0x0a1929, 5, 20);

    // 创建相机
    const camera = new THREE.PerspectiveCamera(60, width / height, 0.1, 1000);
    camera.position.set(0, 5, 10);

    // 创建渲染器
    const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
    renderer.setSize(width, height);
    renderer.setPixelRatio(window.devicePixelRatio);
    mountRef.current.appendChild(renderer.domElement);

    // 创建光源
    const ambientLight = new THREE.AmbientLight(0x404040, 2);
    scene.add(ambientLight);
    const directionalLight = new THREE.DirectionalLight(0xffffff, 1);
    directionalLight.position.set(5, 10, 5);
    scene.add(directionalLight);

    // 创建3D数据对象
    const sphere = createTokenSphere(data.totalTokens);
    scene.add(sphere);

    const bars = createRoleBars(data.roleStats);
    scene.add(bars);

    // 动画循环
    const animate = () => {
      requestAnimationFrame(animate);
      sphere.rotation.y += 0.005;
      bars.children.forEach((bar, i) => {
        bar.rotation.y += 0.01 * (i + 1);
      });
      renderer.render(scene, camera);
    };
    animate();

    // 鼠标交互
    const controls = new OrbitControls(camera, renderer.domElement);
    controls.enableDamping = true;
```

```
controls.dampingFactor = 0.05;

return () => {
  renderer.dispose();
  mountRef.current?.removeChild(renderer.domElement);
};
}, [data]);

return <div ref={mountRef} className="dashboard-3d" />;
};
```

Dashboard3D组件支持鼠标拖拽旋转视角、滚轮缩放和悬停高亮。3D球体的表面纹理根据Token消耗量动态生成颜色渐变（从蓝到红表示消耗等级）。立体柱状图的高度对应各AI角色的Token消耗量，柱体颜色按角色主题色区分。组件还包含粒子效果，当有新的API调用时，粒子从对应数据对象发射，营造实时活动感。

8.5 数据可视化SVG组件库设计

平台自研了一套轻量级SVG数据可视化组件库，包含十余种图表类型。所有图表组件基于SVG绘制，不依赖第三方图表库，具有体积小、定制灵活、性能好的优势。

8.5.1 组件库概览

SVG图表组件库包含以下图表类型：

雷达图（RadarChart）——多维度数据展示，支持数据区域填充和维度标签。用于探路者AI的市场分析面板。

漏斗图（FunnelChart）——漏斗形状的分层展示，支持层间占比和转化率标注。用于探路者AI的机会评估面板。

矩阵图（MatrixChart）——二维矩阵散点图，支持象限划分和节点详情。用于探路者AI的竞品调研面板和军师AI的风险矩阵面板。

时间线（Timeline）——垂直或水平时间轴，支持事件节点和描述。用于探路者AI的趋势洞察面板。

趋势曲线（TrendLine）——平滑曲线图，支持多数据系列和数据点标注。用于探路者AI的趋势洞察面板。

面积图（AreaChart）——带面积填充的曲线图，支持渐变填充。用于用量统计的趋势展示。

柱状图（BarChart）——垂直或水平柱状图，支持分组和堆叠。用于探路者AI的市场分析面板。

环形图（DonutChart）——环形占比图，支持中心文本和多段展示。用于用量统计的模型分布面板。

仪表盘（GaugeChart）——仪表表盘图，支持指针和刻度区间。用于探路者AI的机会评估面板。

进度环（ProgressRing）——环形进度条，支持多环叠加。用于管家AI的进度跟踪面板。

热力计（HeatMeter）——热力计组件，支持渐变色温和数值标注。用于管家AI的统计数据展示。

8.5.2 组件设计规范

```
// SVG图表组件基类接口
interface SVGChartProps {
  width?: number;
  height?: number;
  data: any;
  colors?: string[];
  animate?: boolean;
  responsive?: boolean;
  onElementClick?: (element: ChartElement) => void;
  onElementHover?: (element: ChartElement, event: MouseEvent) => void;
  tooltip?: boolean | TooltipConfig;
  legend?: boolean | LegendConfig;
}

// 所有SVG图表组件继承自基类
abstract class SVGChartBase<P extends SVGChartProps> extends React.Component<P> {
  protected svgRef: React.RefObject<SVGSVGElement>;

  abstract renderChart(): React.ReactNode;
  abstract calculateLayout(): void;
  abstract getTooltipContent(element: ChartElement): string;

  render() {
    return (
      <svg
        ref={this.svgRef}
        width={this.props.width || '100%'}
        height={this.props.height || 300}
        viewBox={`0 0 ${this.props.width || 400} ${this.props.height || 300}`}
      >
        {this.renderChart()}
      </svg>
    );
  }
}
```

SVG图表组件库的设计遵循统一的设计规范：所有组件支持响应式布局（通过viewBox自适应容器尺寸）、支持入场动画（数据加载时的过渡动画）、支持交互（悬停tooltip、点击事件、图例切换）和主题定制（颜色方案、字体大小等可通过props配置）。组件库还提供了统一的颜色方案和间距规范，确保所有图表的视觉一致性。

9 运行设计

本章描述青宸智汇平台的运行环境要求、构建部署流程和微前端集成方案，为系统的部署和运维提供指导。

9.1 运行环境

平台的运行环境分为开发环境、测试环境和生产环境三个层次，各环境配置如下：

9.1.1 开发环境

开发环境用于日常开发调试，配置要求如下：

操作系统：Windows 11 Professional（推荐）或 macOS Sonoma 14+或 Ubuntu 22.04 LTS。开发团队成员使用统一的操作系统和开发工具版本，避免环境差异导致的问题。

开发工具：Visual Studio Code 或 Trae IDE。推荐安装的VS Code扩展包括：ESLint（代码检查）、Prettier（代码格式化）、TypeScript Vue Plugin（TS支持）、Tailwind CSS IntelliSense（样式提示）、GitLens（Git增强）等。

运行时环境：Node.js v20.10.0 LTS，包管理器使用npm 10+或pnpm 8+。建议使用nvm（Node Version Manager）管理Node.js版本，确保团队成员使用统一的Node版本。

浏览器：Google Chrome 120+（开发调试推荐使用最新版Chrome浏览器，安装React Developer Tools和Redux DevTools扩展）。

版本控制：Git 2.40+，代码托管于GitHub或GitLab。遵循Git Flow分支管理策略，主分支为main（生产代码），开发分支为develop，功能分支为feature/*，修复分支为hotfix/*。

9.1.2 测试环境

测试环境用于功能测试、集成测试和性能测试，配置要求如下：

服务器：2核CPU、4GB内存、50GB SSD存储的云服务器（如阿里云ECS或腾讯云CVM）。

操作系统：Ubuntu 22.04 LTS Server。

部署方式：使用Docker容器化部署，前端静态资源通过Nginx提供，AI服务通过反向代理转发到后端API。

数据库：PostgreSQL 15（存储用户数据、对话历史等），Redis 7（缓存和会话管理）。

监控：使用Grafana + Prometheus进行系统监控，包括CPU、内存、网络和响应时间等指标。

9.1.3 生产环境

生产环境面向最终用户，配置要求如下：

CDN：前端静态资源部署于CDN（如阿里云CDN或Cloudflare），确保全国范围内的快速访问。CDN配置缓存策略（HTML文件不缓存，JS/CSS文件长期缓存，通过文件名hash实现缓存更新）。

服务器：根据用户规模配置。初期支持单节点部署（4核CPU、8GB内存），后期通过负载均衡实现多节点横向扩展。

负载均衡：使用Nginx或云负载均衡器（如阿里云SLB）进行流量分发，支持健康检查和自动故障转移。

HTTPS：全站启用HTTPS，使用Let's Encrypt或云服务商提供的SSL证书。HTTP请求自动重定向到HTTPS。

备份：数据每日自动备份，保留最近30天的备份记录。备份存储于异地（跨可用区或跨地域），确保灾难恢复能力。

9.2 构建部署流程

平台的构建部署流程包含代码提交、自动构建、自动测试和自动部署四个阶段，采用CI/CD（持续集成/持续部署）流水线实现自动化。

9.2.1 构建流程

构建流程使用Vite进行前端代码编译和打包。具体步骤如下：

第一步，依赖安装——执行`npm install --frozen-lockfile`，根据lock文件安装精确版本的依赖包，确保构建环境的一致性。

第二步，代码检查——执行ESLint和TypeScript编译检查，确保代码质量符合规范。ESLint检查代码风格和潜在问题，TypeScript编译检查类型错误。检查不通过将中断构建流程。

第三步，单元测试——执行Jest或Vitest运行单元测试，确保代码功能正确。测试覆盖率不低于80%，核心模块覆盖率不低于90%。

第四步，生产构建——执行`vite build`命令，Vite使用Rollup进行代码打包。构建过程包括代码分割（vendor和业务代码分离）、Tree Shaking（移除未使用代码）、代码压缩（Terser压缩JS，cssnano压缩CSS）和资源优化（图片压缩、字体子集化等）。

第五步，产物处理——构建产物输出到dist目录，包含index.html、JS chunks、CSS文件和静态资源。通过文件名hash实现缓存控制，每次构建生成新的hash文件名。

```
# 构建脚本示例 (package.json scripts)
{
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "build:analyze": "vite build --mode analyze",
    "preview": "vite preview",
    "lint": "eslint src --ext .ts,.tsx",
    "test": "vitest",
    "test:coverage": "vitest --coverage",
    "build:prod": "npm run lint && npm run test && npm run build"
  }
}
```

9.2.2 部署流程

部署流程使用CI/CD流水线（如GitHub Actions或GitLab CI）实现自动化部署。具体步骤如下：

代码提交触发——开发者将代码合并到main分支后，自动触发CI/CD流水线。流水线从代码仓库拉取最新代码，开始构建流程。

自动构建——CI/CD流水线执行上述构建流程的五个步骤，生成构建产物。如果构建失败，流水线中断并通知开发者。

自动测试——构建成功后，流水线运行端到端测试（E2E Test），确保构建产物功能正确。E2E测试使用Playwright或Cypress框架。

自动部署——测试通过后，流水线将构建产物部署到目标环境。部署方式根据环境不同：测试环境直接部署，生产环境通过蓝绿部署或滚动部署策略进行。

部署验证——部署完成后，流水线执行健康检查（发送HTTP请求验证服务可用性），确保部署成功。如果健康检查失败，自动回滚到上一个版本。

```
# GitHub Actions CI/CD 配置示例 (.github/workflows/deploy.yml)
name: Build and Deploy

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm run test:coverage
      - run: npm run build
      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Download artifacts
        uses: actions/download-artifact@v4
        with:
```

```
    name: dist
    path: dist/
  - name: Deploy to CDN
    run: |
      # 上传到CDN/OSS
      aws s3 sync dist/ s3://bucket-name/ --delete
  - name: Health check
    run: |
      curl -f https://app.example.com/ || exit 1
```

9.3 微前端集成

平台采用qiankun微前端架构，主应用作为基座，各AI智能体模块作为子应用独立部署。微前端集成的关键技术方案如下：

9.3.1 主应用配置

主应用负责全局状态管理、路由分发和子应用生命周期管理。主应用在启动时注册所有子应用，根据路由匹配自动加载对应的子应用。主应用的入口文件配置qiankun的start方法，启用沙箱模式（sandbox: true）和预加载（prefetch: true）。

主应用通过initGlobalState方法创建全局状态通信通道，子应用通过onGlobalStateChange监听全局状态变化，通过setGlobalState更新全局状态。全局状态包括当前用户信息、当前AI角色和全局配置等。

9.3.2 子应用配置

子应用需要导出三个生命周期函数：bootstrap（子应用首次加载时调用，执行一次性的初始化）、mount（子应用每次挂载时调用，执行DOM渲染和事件绑定）和unmount（子应用卸载时调用，执行清理和资源释放）。

子应用的Vite配置需要设置base路径为子应用的部署路径，确保静态资源的正确加载。子应用的入口HTML需要配置允许跨域访问的CORS头部（Access-Control-Allow-Origin）。

```
// 子应用生命周期导出 (src/main.ts)
export async function bootstrap() {
  console.log('子应用启动');
}

export async function mount(props) {
  const container = props.container;
  ReactDOM.createRoot(container.querySelector('#root')).render(
    <React.StrictMode>
      <App />
    </React.StrictMode>
  );
}

export async function unmount(props) {
  // 清理逻辑
}
```

```
const container = props.container;
const root = container.querySelector('#root');
if (root) {
  ReactDOM.createRoot(root).unmount();
}
}
```

9.3.3 样式隔离

qiankun提供两种样式隔离方案：严格沙箱（StrictSandbox）和实验性沙箱（ExperimentalSandbox）。严格沙箱通过Shadow DOM实现样式隔离，确保子应用样式不影响主应用和其他子应用。实验性沙箱通过动态CSS作用域实现样式隔离，兼容性更好但隔离效果略弱。

平台采用严格沙箱模式，确保各子应用样式的完全隔离。同时，主应用通过CSS变量（CSS Custom Properties）传递主题色彩等全局样式参数，子应用通过读取CSS变量实现主题适配。

9.3.4 通信机制

主应用与子应用之间的通信通过qiankun的initGlobalState API实现。通信消息采用JSON格式，包含消息类型和数据内容。主应用负责维护全局状态的权威版本，子应用通过监听全局状态变化获取最新状态，通过发送消息请求主应用更新全局状态。

通信场景包括：用户登录状态同步（主应用登录后通知子应用）、AI角色切换同步（主应用切换角色后通知子应用加载对应面板）和主题切换同步（主应用切换主题后通知子应用更新样式）。

10 出错处理设计

本章描述青宸智汇平台的错误处理设计方案，包括网络错误处理、认证过期处理、AI服务异常处理和前端错误边界设计。完善的错误处理机制是保证系统稳定性和用户体验的关键。

10.1 网络错误处理

网络错误是前端应用最常见的错误类型，包括网络超时、连接中断、DNS解析失败和服务器不可达等。平台的网络错误处理策略如下：

10.1.1 请求超时处理

HTTP请求封装层为每个请求设置默认超时时间（普通请求30秒，文件上传5分钟，AI流式请求无超时限制）。请求超时后，封装层自动取消请求（通过AbortController），并向用户显示友好的超时提示信息。用户可以选择重试请求或取消操作。

```
// 请求超时处理
function fetchWithTimeout(url: string, options: RequestInit, timeout = 30000): Promise<Response> {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeout);

  return fetch(url, {
    ...options,
    signal: controller.signal,
  }).finally(() => {
    clearTimeout(timeoutId);
  });
}

// 重试机制
async function fetchWithRetry(
  url: string,
  options: RequestInit,
  retries = 3,
  delay = 1000
): Promise<Response> {
  try {
    return await fetchWithTimeout(url, options);
  } catch (error) {
    if (retries <= 0) throw error;
    await new Promise(resolve => setTimeout(resolve, delay));
    return fetchWithRetry(url, options, retries - 1, delay * 2);
  }
}
```

10.1.2 网络状态监控

平台通过navigator.onLine API和online/offline事件监听网络状态变化。当检测到网络断开时，自动显示全局网络断开提示条，提示用户检查网络连接。网络恢复后，自动隐藏提示条并重新发送因网络中断而失败的请求。

对于AI流式响应，网络中断会导致流式连接断开。平台在检测到流式连接断开时，自动尝试重连（最多重试3次），重连成功后从断点继续接收流式数据。如果重连失败，将已接收的部分内容保留显示，并提示用户网络中断。

10.1.3 离线缓存

平台使用Service Worker实现离线缓存功能。Service Worker缓存核心页面和静态资源，在网络不可用时，用户仍可以访问已缓存的页面和数据。对于AI对话功能，平台在本地缓存最近的对话历史，用户在离线状态下可以查看历史对话（但无法发送新消息）。

10.2 认证过期处理

JWT Token具有过期时间（默认2小时），Token过期后需要通过Refresh Token刷新或引导用户重新登录。平台的认证过期处理流程如下：

10.2.1 Token自动刷新

HTTP请求封装层在每次请求前检查Token的剩余有效期。如果Token将在5分钟内过期，封装层自动在后台发送Refresh Token请求，获取新的Access Token和Refresh Token。刷新过程对用户透明，不影响当前请求的发送。

```
// Token自动刷新
let isRefreshing = false;
let refreshSubscribers: Array<(token: string) => void> = [];

async function ensureValidToken(): Promise<string | null> {
  const auth = useAuthStore.getState();
  if (!auth.token) return null;

  // 解析Token获取过期时间
  const payload = JSON.parse(atob(auth.token.split('.')[1]));
  const expiresAt = payload.exp * 1000;
  const now = Date.now();
  const fiveMinutes = 5 * 60 * 1000;

  // 如果Token将在5分钟内过期，刷新Token
  if (expiresAt - now < fiveMinutes) {
    if (!isRefreshing) {
      isRefreshing = true;
      try {
        const response = await refreshToken(auth.refreshToken);
        useAuthStore.getState().updateToken(response.token, response.refreshToken);
        refreshSubscribers.forEach(cb => cb(response.token));
      }
    }
  }
}
```

```
        refreshSubscribers = [];  
        return response.token;  
    } catch {  
        useAuthStore.getState().logout();  
        redirectToLogin();  
        return null;  
    } finally {  
        isRefreshing = false;  
    }  
} else {  
    // 如果正在刷新，等待刷新完成  
    return new Promise(resolve => {  
        refreshSubscribers.push(resolve);  
    });  
}  
}  
  
return auth.token;  
}
```

10.2.2 401错误处理

当后端返回401（Unauthorized）错误时，表示Token已过期或无效。HTTP请求封装层的错误处理拦截器捕获401错误，尝试通过Refresh

Token刷新Token。如果刷新成功，自动重试原始请求。如果刷新失败（Refresh Token也过期），清除用户登录状态，跳转到登录页面，并在登录页面显示“会话已过期，请重新登录”的提示。

10.2.3 并发请求处理

当多个请求同时收到401错误时，只需要发送一次Refresh Token请求。平台使用Promise队列机制确保多个401错误只触发一次Token刷新。第一个401错误发起刷新请求，后续的401错误等待刷新完成后使用新Token重试。这种机制避免了重复刷新导致的Token失效问题。

10.3 AI服务异常处理

AI服务是平台的核心依赖，AI服务异常将直接影响用户体验。平台的AI服务异常处理策略如下：

10.3.1 AI服务不可用

当AI服务不可用时（如API返回503服务不可用、502网关错误或连接超时），平台显示友好的错误提示，建议用户稍后重试。同时提供“切换模型”选项，允许用户切换到备用AI模型（如从GPT-4o切换到GPT-4o-mini）。

10.3.2 流式响应中断

AI流式响应过程中可能因网络中断、服务器错误或超时而中断。平台在检测到流式响应中断时，保留已接收的部分内容，在内容末尾添加“[响应中断]”标记。用户可以选择“继续生成”（发送续写请求）或“重新生成”（重新发送完整请求）。

10.3.3 内容安全过滤

AI服务可能因内容安全策略拒绝生成某些内容（返回content_filter错误）。平台在收到内容安全拒绝时，显示提示信息“AI服务因内容安全策略拒绝了此请求，请修改后重试”。同时建议用户调整提问方式或更换AI角色。

10.3.4 额度限制处理

当用户的API调用额度耗尽时（返回429 Too Many Requests或quota_exceeded错误），平台显示额度不足提示，建议用户查看用量统计页面了解消耗情况，或联系管理员增加额度。

10.4 前端错误边界

React错误边界（Error Boundary）是React 16+提供的错误处理机制，用于捕获子组件树中的JavaScript错误，防止错误扩散导致整个应用崩溃。平台的错误边界设计如下：

10.4.1 全局错误边界

平台在应用根组件外层包裹全局错误边界组件。全局错误边界捕获应用中未被局部错误边界处理的错误，显示全局错误页面（包含错误信息、错误代码和“返回首页”按钮），同时将错误信息上报到错误监控系统（如Sentry）。

```
// 全局错误边界
class GlobalErrorBoundary extends React.Component<
  { children: React.ReactNode },
  { hasError: boolean; error: Error | null }
> {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error: Error) {
    return { hasError: true, error };
  }

  componentDidCatch(error: Error, errorInfo: React.ErrorInfo) {
    // 上报错误到监控系统
    Sentry.captureException(error, { extra: errorInfo });
    console.error('Global error caught:', error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return (
        <div className="error-page">
          <Result
            status="500"
            title="应用发生错误"
            subTitle={this.state.error?.message || '未知错误'}
            extra={[
              <Button type="primary" key="home" onClick={() => window.location.href = '/'}>
                返回首页
              </Button>
            ]}
          </Result>
        </div>
      );
    }
  }
}
```

```
        </Button>,
        <Button key="retry" onClick={() => window.location.reload()}>
          重新加载
        </Button>,
      ]}
    />
  </div>
);
}
return this.props.children;
}
```

10.4.2 局部错误边界

除了全局错误边界，平台在关键功能区域（如AI对话面板、思维导图编辑器、3D仪表盘等）设置局部错误边界。局部错误边界捕获特定区域的错误，显示区域级错误提示（包含错误信息和“重试”按钮），不影响其他区域的正常使用。这种分层错误边界策略确保了局部错误不会扩散到全局，最大程度保证了应用的可用性。

10.4.3 异步错误处理

React错误边界无法捕获异步错误（如Promise rejection、setTimeout回调错误等）。平台通过以下方式处理异步错误：

全局Promise rejection监听——通过window.addEventListener("unhandledrejection", handler)捕获未处理的Promise rejection，显示全局错误提示。

全局error事件监听——通过window.addEventListener("error", handler)捕获未捕获的JavaScript错误，包括资源加载失败等。

React组件内try-catch——在异步操作中显式使用try-catch捕获错误，通过setState设置错误状态，在组件中渲染错误提示。

11 安全设计

本章描述青宸智汇平台的安全设计方案，包括身份认证（JWT）、API密钥管理和数据安全。安全设计是保障平台用户数据和系统资源安全的基础。

11.1 身份认证（JWT）

平台采用JWT（JSON Web Token）进行用户身份认证。JWT是一种无状态的认证方案，服务端不需要存储会话信息，适合分布式和微前端架构。

11.1.1 JWT认证流程

JWT认证流程包含以下步骤：用户登录时，前端将用户名和密码发送到认证服务接口（POST /api/auth/login）。认证服务验证用户名密码后，生成Access Token和Refresh Token。Access Token包含用户ID、角色和过期时间等声明（Claims），使用HS256算法签名。Refresh Token是一个随机字符串，存储在服务端数据库中，用于刷新Access Token。

前端收到Token后，将Access Token存储在内存中（不存储在localStorage中以防止XSS攻击），将Refresh Token存储在HttpOnly Cookie中（防止JavaScript读取）。后续请求在Authorization头部中携带Access Token。服务端验证Token签名和过期时间，通过后处理请求。

11.1.2 Token安全策略

Access Token有效期设置为2小时，Refresh Token有效期设置为7天。Token使用HS256算法签名，密钥存储在服务端环境变量中，不硬编码在代码中。

Token吊销机制——当用户主动登出或修改密码时，服务端将对应的Refresh Token标记为无效，使后续的Token刷新请求失败。同时，将Access Token加入黑名单（存储在Redis中，设置与Token相同的过期时间），使正在使用的Access Token立即失效。

Token防重放攻击——每个Token包含一个唯一的jti（JWT ID）声明，服务端记录已使用的jti，防止Token被重放攻击。

11.2 API密钥管理

平台需要管理多个外部服务的API密钥，包括AI模型服务（OpenAI API Key）、行业数据服务和第三方工具服务等。API密钥管理设计如下：

11.2.1 密钥存储

API密钥不存储在前端代码或环境变量中，而是存储在服务端的密钥管理服务（如AWS Secrets Manager或HashiCorp Vault）中。前端通过后端代理访问外部API，后端在请求中注入API密钥，前端永远不接触原始密钥。

密钥存储采用加密方式，使用AES-256-GCM算法加密密钥内容。加密密钥（Master Key）存储在硬件安全模块（HSM）或密钥管理服务中，不与应用代码存储在同一位置。

11.2.2 密钥轮换

API密钥定期轮换（建议每90天一次），降低密钥泄露的风险。密钥轮换通过自动化脚本执行，新密钥生效后旧密钥保留24小时（宽限期），确保正在进行的请求不受影响。宽限期过后，旧密钥自动失效。

密钥泄露应急响应——当检测到密钥可能泄露时（如异常API调用量激增），立即吊销当前密钥并生成新密钥，同时审计泄露期间的API调用记录，评估影响范围。

11.2.3 用量限制

平台对每个用户和每个API密钥设置用量限制（Rate Limiting），防止单个用户或密钥过度消耗资源。用量限制通过令牌桶算法实现，每秒允许的请求数和每日允许的Token消耗量根据用户等级配置。超出限制时返回429错误，提示用户稍后重试或升级套餐。

11.3 数据安全

平台的数据安全设计涵盖传输安全、存储安全和隐私保护三个方面。

11.3.1 传输安全

全站启用HTTPS（TLS 1.3），所有HTTP请求自动重定向到HTTPS。TLS证书使用Let's Encrypt或云服务商提供的证书，定期自动续期。

API请求和响应使用JSON格式，敏感字段（如密码、Token等）在传输前进行客户端加密（使用RSA公钥加密），服务端使用RSA私钥解密。这种端到端加密确保即使传输过程被截获，敏感数据也无法被解密。

11.3.2 存储安全

前端本地存储的安全策略：localStorage仅存储非敏感数据（如用户偏好设置、主题配置等），不存储Token和密码等敏感信息。sessionStorage用于存储会话相关的临时数据，浏览器关闭后自动清除。IndexedDB用于存储较大的本地数据（如离线缓存的对话历史），设置过期时间定期清理。

后端数据库的安全策略：用户密码使用bcrypt算法加盐哈希存储，不存储明文密码。对话历史和用户信息等敏感数据在数据库中加密存储（使用AES-256-GCM算法）。数据库访问使用最小权限原则，应用使用只读账号查询数据，仅管理操作使用读写账号。

11.3.3 隐私保护

平台遵循《个人信息保护法》等隐私法规要求，用户隐私保护设计如下：

数据最小化——仅收集必要的用户信息（用户名、邮箱、学校、专业），不收集与功能无关的个人信息。

用户知情同意——注册时明确告知用户数据收集和使用目的，用户勾选同意后方可注册。隐私政策页面详细说明数据处理方式。

数据删除权——用户可以随时删除自己的账户和所有数据。删除操作在30天内完成（包括数据库记录、备份文件和缓存数据），删除后不可恢复。

数据导出权——用户可以导出自己的所有数据（对话历史、项目数据、个人信息等），导出格式为JSON或CSV。

11.3.4 XSS防护

跨站脚本攻击（XSS）是前端应用的主要安全威胁。平台的XSS防护策略包括：

输入过滤——所有用户输入在渲染前进行HTML转义，防止恶意脚本注入。React默认对JSX表达式中的内容进行HTML转义，但使用dangerouslySetInnerHTML时需要手动过滤。

CSP策略——通过Content-Security-Policy头部设置内容安全策略，限制脚本来源（仅允许同源和可信CDN的脚本），禁止内联脚本和eval。

HttpOnly

Cookie——认证Cookie设置为HttpOnly，防止JavaScript读取，降低XSS攻击窃取Cookie的风险。

11.3.5 CSRF防护

跨站请求伪造（CSRF）防护通过双重提交Cookie方案实现：服务端在登录时生成CSRF Token，存储在Cookie中。前端在发送请求时从Cookie中读取CSRF Token，添加到请求头（X-CSRF-Token）。服务端验证请求头中的Token与Cookie中的Token是否一致。

12 附录

本章附录提供平台的技术栈版本清单和第三方依赖列表，作为开发参考。

12.1 技术栈版本清单

以下为平台package.json中定义的核心依赖列表及其版本信息。

```
// package.json 核心依赖
{
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-router-dom": "^6.22.0",
    "typescript": "^6.0.0",

    "antd": "^6.0.0",
    "@ant-design/icons": "^6.0.0",
    "@ant-design/pro-components": "^2.7.0",

    "zustand": "^5.0.0",
    "immer": "^10.1.0",

    "qiankun": "^2.10.0",

    "three": "^0.165.0",
    "@react-three/fiber": "^8.16.0",
    "@react-three/drei": "^9.108.0",

    "echarts": "^5.5.0",

    "axios": "^1.7.0",
    "dayjs": "^1.11.0",

    "marked": "^12.0.0",
    "highlight.js": "^11.10.0",

    "lodash-es": "^4.17.21",
    "clsx": "^2.1.0",

    "dompurify": "^3.1.0",
    "crypto-js": "^4.2.0"
  },

  "devDependencies": {
    "vite": "^8.0.0",
    "@vitejs/plugin-react": "^4.3.0",
    "vite-plugin-qiankun": "^1.0.0",

    "typescript": "^6.0.0",
  },
}
```

```
"@types/react": "^19.0.0",
"@types/react-dom": "^19.0.0",
"@types/three": "^0.165.0",
"@types/lodash-es": "^4.17.0",

"eslint": "^9.0.0",
"@typescript-eslint/parser": "^7.0.0",
"@typescript-eslint/eslint-plugin": "^7.0.0",
"eslint-plugin-react": "^7.35.0",
"eslint-plugin-react-hooks": "^4.6.0",
"prettier": "^3.3.0",

"vitest": "^2.0.0",
"@testing-library/react": "^16.0.0",
"@testing-library/jest-dom": "^6.4.0",
"@playwright/test": "^1.45.0",

"sass": "^1.77.0",
"postcss": "^8.4.0",
"autoprefixer": "^10.4.0"
}
}
```

以上依赖版本基于2026年8月的最新稳定版本。实际开发中可能根据项目需求进行版本调整，建议在package.json中锁定具体版本号（不使用^或~前缀），确保构建的一致性。

12.2 第三方依赖说明

平台使用的第三方依赖按功能分类说明如下：

12.2.1 核心框架依赖

React 19——UI框架，提供组件化开发和Hooks机制。react-dom提供React的DOM渲染器。react-router-dom提供客户端路由管理。

TypeScript 6——JavaScript超集，提供静态类型系统。所有源代码使用TypeScript编写，编译为JavaScript运行。

12.2.2 UI组件库依赖

Ant Design 6——企业级UI组件库，提供按钮、表单、表格、弹窗等40余种组件。@ant-design/icons提供图标库。@ant-design/pro-components提供高级业务组件（ProTable、ProForm等）。

12.2.3 状态管理依赖

Zustand 5——轻量级状态管理库，提供简洁的API和Hook模式。Immer 10——不可变数据管理库，通过immer中间件集成到Zustand。

12.2.4 微前端依赖

qiankun 2.10——微前端框架，提供子应用注册、加载、沙箱隔离和通信机制。vite-plugin-qiankun——Vite插件，简化qiankun子应用的Vite配置。

12.2.5 3D可视化依赖

Three.js 0.165——WebGL 3D图形库，提供3D场景、相机、光源、材质和动画等能力。@react-three/fiber——React renderer for Three.js，以React组件方式使用Three.js。@react-three/drei——Three.js辅助工具库，提供常用3D组件和工具函数。

12.2.6 数据可视化依赖

ECharts 5.5——百度开源数据可视化库，支持折线图、柱状图、饼图、雷达图等图表类型。平台在部分复杂数据展示场景中使用ECharts，其余场景使用自研SVG图表组件库。

12.2.7 网络通信依赖

Axios 1.7——HTTP请求库，提供请求拦截、响应拦截、错误处理和请求取消等功能。Fetch API（浏览器原生）用于流式响应处理。dayjs 1.11——轻量级日期处理库，用于日期格式化和计算。

12.2.8 文档渲染依赖

Marked 12.0——Markdown解析器，将AI返回的Markdown格式文本解析为HTML。highlight.js 11.10——代码语法高亮库，为代码块添加语法高亮。DOMPurify 3.1——HTML净化库，过滤Markdown渲染产生的HTML中的恶意脚本，防止XSS攻击。

12.2.9 工具库依赖

lodash-es 4.17——JavaScript实用工具库，提供防抖、节流、深拷贝、合并等工具函数。clsx 2.1——CSS类名组合工具，根据条件动态组合CSS类名。crypto-js 4.2——加密库，用于客户端数据加密（如密码加密、敏感数据传输加密）。

12.2.10 开发工具依赖

Vite 8——前端构建工具，提供开发服务器和生产构建。ESLint 9——代码检查工具，配合TypeScript ESLint插件检查代码质量。Prettier 3.3——代码格式化工具，统一代码风格。Vitest 2——单元测试框架，兼容Jest API。Playwright 1.45——端到端测试框架，支持Chrome、Firefox和Safari浏览器。Sass 1.77——CSS预处理器，支持变量、嵌套和混入等特性。

12.3 源程序规模统计

平台源代码规模统计如下：

源代码总行数：约29561行（含注释和空行）。

源文件总数：148个（含.ts、.tsx、.scss、.css、.json等文件类型）。

TypeScript文件：约120个（.ts和.tsx文件）。

样式文件：约15个（.scss和.css文件）。

配置文件：约13个（.json、.yaml、.conf等文件）。

代码结构按功能模块分布：components目录约45个文件（通用组件和图表组件），modules目录约55个文件（四大AI智能体和工具箱模块），services目录约12个文件（服务层接口），stores目录约8个文件（状态管理），types目录约6个文件（类型定义），utils目录约10个文件（工具函数），其他目录约12个文件。

12.4 修订记录

本设计说明书的修订记录如下：

V1.0（2026年8月）——初始版本。包含完整的系统架构设计、模块设计、接口设计、界面设计、数据结构设计、运行设计、出错处理设计和安全设计。覆盖平台全部功能模块和技术方案。